

一、课程基本信息

- **主讲人:** (Prof. Xu Ruifeng)
- **教学团队:** 包含助教, 并提供联系方式。
- **参考教材:** 列出了多本经典和前沿的中文 NLP 教材, 涵盖了从统计 NLP 到大语言模型的不同阶段。
- **评分标准:**
 - **课程项目 (40%):** 强调系统设计与实现, 考核标准侧重于**性能(70%)**、创新性(20%)和报告演示(10%)。
 - **期末考试 (60%)**
 - **团队合作:** 鼓励学生三人一组进行协作。

二、什么是自然语言处理 (NLP)?

这部分从概念上界定了 NLP 的研究范畴。

- **自然语言 (Natural Language):**
 - **定义:** 人类用于通用交流的语言, 可以是口语、书面语或手语。
 - **多样性:**
 - **多媒体 (Multi-media):** 文本、音频、视频。
 - **多语言 (Multi-lingual):** 全球有超过 150 种语言。
 - **多载体 (Multi-Biological):** 书写文字、口语、手语、盲文等。
 - **历史性 (Historical):** 语言的字、词、语法、发音都随时间演变。
- **自然语言处理 (NLP):**
 - **核心定义:** 研究如何让计算机自动地 **处理 (processing)**、**理解 (understanding)** 和 **生成 (generation)** 人类语言的学科。
 - **核心流程:**
 - **理解 (Understanding):** 将自然语言输入转换为计算机可处理的结构化信息。
 - **生成 (Generation):** 将计算机内部的信息转换为人类可读的自然语言输出。

三、NLP 能做什么? (应用场景)

这部分展示了 NLP 技术的广泛应用, 让学生对该领域有直观的感受。

- **基础应用:**
 - **自动纠错与补全:** 如搜索引擎的输入提示。
 - **信息检索 (Information Retrieval):** 搜索引擎的核心。
 - **机器翻译 (Machine Translation):** 如谷歌翻译。
- **进阶应用:**
 - **信息抽取 (Information Extraction):** 从非结构化文本中提取结构化信息 (如姓名、地址)。
 - **问答系统/智能助手:** 如 Siri, IBM Watson, 能够理解用户意图并提供服务。
 - **情感分析/评论挖掘:** 从用户评论中提取关键词和情感倾向。
- **前沿应用:**
 - **对话系统/辩论 AI:** 如 IBM 的 Project Debater, 展示了复杂的论点构建和反驳能力。

四、为什么 NLP 很难? (核心挑战)

这部分深入探讨了 NLP 的内在困难, 核心在于语言的“**歧义性 (Ambiguity)**”。

- **语言的复杂性:** 存在海量的词汇、复杂的语法规则和大量的例外情况。

- **歧义性 (Ambiguity):** 同一个表达在不同层面可能产生多种解释。
 - **语音层面 (Acoustic Level):** 同音异形词, 如 "like your mother" vs "lie cured your mother"。
 - **词法层面 (Word Level):** 中文分词歧义 (“治理解放大道路面积水”)、一词多义 (“意思”)、网络用语 (“河蟹”)。
 - **句法层面 (Syntactic Level):** 同一句子可以有不同的语法结构, 导致含义不同。
 - **语义层面 (Semantic Level):** 词语本身有多重含义 (如 "mother" 可以指母亲, 也可以指醋母)。
 - **语篇层面 (Discourse Level):** 指代消解的歧义 ("she" 指的是谁)。
- **知识瓶颈 (Knowledge Bottleneck):** 真正的语言理解需要 **语言知识** 和 **世界知识**。如何将这些知识有效地教给计算机是核心难题。
 - **符号主义 (Rationalism):** 尝试将所有规则和知识手动编码, 但难度巨大且无法覆盖所有情况。
 - **统计/经验主义 (Empiricism):** 从大量语料中自动学习语言规律, 是现代 NLP 的主流方法。

五、NLP 发展简史 (技术演进脉络)

这部分通过一个时间线, 宏观地展示了 NLP 从诞生到现在的技术范式变迁。

- **1940s-1950s (萌芽与狂热):** 受密码学启发, 机器翻译起步。以乔姆斯基的生成语法为代表的 **符号主义** 占据主导。
- **1960s (寒冬期):** ALPAC 报告指出机器翻译进展甚微, 导致经费削减, 研究进入低谷。
- **1970s-1980s (复苏期):** 理论研究持续, 出现了一些专家系统。**统计方法** 开始在语音识别等领域取得成功。
- **1990s (统计方法主导):** IBM 的统计机器翻译系统和 Web 的兴起, 使基于数据的统计方法成为主流。
- **2000s (数据为王):** “给我足够的数据, 我就能...” 成为时代强音。NLP 技术在搜索、数据挖掘等领域广泛商业化。
- **2010s (深度学习时代):**
 - **词表示:** Word2Vec 等 **分布式表示 (词向量)** 解决了词义的向量化问题。
 - **模型架构:** CNN, RNN, LSTM, Seq2Seq+Attention 等模型在各项任务中取得突破。
- **2020s (大模型/基础模型时代):**
 - **预训练语言模型:** GPT、BERT 等模型统一了 NLP 的范式。
 - **模型规模化:** GPT-3、PaLM 等模型参数量和能力急剧增长。
 - **指令学习与对齐:** InstructGPT 和 ChatGPT 通过 **RLHF** 实现了与人类意图的对齐, 引爆了生成式 AI 的浪潮。

六、课程设计与总结

- **NLP 技术路线:** 课程将涵盖符号/知识、统计、神经网络三大技术范式。
- **课程结构:** 分为 **预备知识**、**基础技术**、**应用**、**大语言模型** 四大部分。
- **学术资源:** 介绍了 NLP 领域的顶级学术组织(ACL, CIPS)和会议(ACL, EMNLP, COLING 等)。

Lecture 2: Linguistics Foundations - 核心知识点总结

第一部分：汉字基础 (Chinese Characters)

1. 汉字特性:
 - 非字母符号 (Non-alphabetic), 是 表意文字 (Ideographic character)。
2. 汉字构成四法 (Character Information):
 - 象形 (Pictograph): 描绘事物形状。例：日、月、山。数量少。
 - 指示 (Ideographic): 抽象符号表示意义。例：上、下。数量很少。
 - 会意 (Compound indicative): 组合已有字符的意义。例：人+言=信。数量很少。
 - 形声 (Semantic-phonetic compounds): **最重要**, 占比超过 90%。一个部分表意 (形旁), 一个部分表音 (声旁)。例：请、情、清、晴 (声旁都是“青”)。
3. 汉字信息表示 (Glyph-vectors):
 - 一种现代技术, 利用汉字的字形图像 (glyph images), 包括其历史形态 (如金文、篆书) 来学习字符的向量表示, 以捕捉更丰富的语义信息。
4. 汉字处理的挑战:
 - 数量巨大 (Large Number): 《康熙字典》收录 47,035 字, 《中华字海》收录 87,019 字。但常用字覆盖率高: 1000 字覆盖 92%, 3000 字覆盖 99% 的书面材料。
 - 繁简转换 (Traditional/Simplified): 并非一一对应。
 - 例 1: 乾 (gān, dry) 和 幹 (gàn, to do) 都简化为 干。
 - 例 2: 遊 (travel) 和 游 (swim) 都简化为 游。
 - 异体字 (Variant Character): 同音同义但字形不同。例：回、同、囧、廻。
 - 方言字 (Dialectal Character): 在特定方言中使用的字。例：粤语的“冇”(没有)。
5. 汉字编码标准 (Character Encoding Standards) - 【高频考点】
 - GB (国标): 主要用于简体中文。
 - GB2312: 早期标准, 包含约 7000 汉字。
 - GBK: GB2312 的扩展, 支持繁体字和更多字符。
 - GB18030: 最新强制标准, 取代所有旧版 GB, 支持简繁体。
 - Big5 (大五码): 主要用于繁体中文 (台湾、香港地区)。
 - Unicode (统一码): 国际行业标准, 旨在统一全球所有字符。
 - UTF-8: 变长编码, ASCII 用 1 字节, 汉字通常用 3 字节。是目前最通用的编码方式。
 - UCS-2 / UTF-16: 2 或 4 字节编码。
 - 汉字统一 (Han Unification): Unicode 将不同地区的同意、同源汉字 (简、繁、日、韩) 赋予同一个编码点。
 - 编码对比示例 (关键): 自然语言处理
 - GB2312: D7 D4 C8 BB D3 EF D1 D4 B4 A6 C0 ED
 - Big5: A6 D8 B5 4D BB 79(語) A8 A5 B3 A2(處) B2 7A
 - UTF-8: E8 87 AA E7 84 B6 E8 AF AD E8 A8 80 E5 A4 84 E7 90 86
6. 基于字的处理 (Character-based Processing):
 - 直接以“字”为单位进行 NLP 任务, 如音译检测 (Transliteration)、词汇情感猜测、新词发现等。

第二部分：词素与词 (Morphemes and Words)

1. 词素 (Morpheme - 词素):

- 定义: 最小的有意义的语言单位。
- 分类 (以英文为例):
 - 自由词素 (Free Morpheme): 可独立成词。例: happy。
 - 粘着词素 (Bound Morpheme): 必须依附于其他词素。例: un-, -ness。
- 中文词素: 通常一个汉字代表一个音节和一个词素。例: “葡萄”是多字词素。

2. 词 (Word - 词):

- 定义: 可独立运用的最小语言单位, 由一个或多个词素构成。
- 中文词的构成法 (Word Formation) - **【核心考点】**
 - 复合 (Compounding):
 - 双音节词 (Disyllabic):
 - 偏正型 (Modified): 热心, 水手 (修饰+中心语)
 - 联合型 (Coordinative): 报告, 声音 (意义相近/相关)
 - 动宾型 (Verb-Object): 放心, 司机 (动作+对象)
 - 主谓型 (Subject-Predicate): 地震, 心疼 (主语+谓语)
 - 动补型 (Verb-Complement): 进来, 进去 (动作+补充结果/方向)
 - 三音节词 (Tri-syllabic): 存在层次结构, 可能产生歧义。
 - 例: (口香)(糖) vs. (开)(玩笑); 大/学生 vs. 大学/生。
 - 四音节词/成语 (Quad-syllabic): 结构固定, 如 花言巧语 (同义词交织), 大同小异 (反义词交织)。
 - 派生 (Affixation):
 - 前缀 (Prefix): 第一, 老公
 - 后缀 (Suffix): 桌子, 物理学, 吃了, 吃着
 - 重叠 (Reduplication):
 - AABB: 漂漂亮亮
 - ABAB: 商量商量
 - 离合 (Ionization): 动宾结构的词可以被拆分使用。
 - 例: 理发 -> 理一个发 / 发理了吗?

第三部分: 汉语处理的挑战 (Challenges in Chinese Processing)

1. 缺乏形态标记 (Few Morphological Markings):

- 动词无时态变化: 我昨天是学生, 我现在是学生, 我将来是学生 (动词 “是” 不变)。
- 名词无单复数/格/性变化:
 - 我的书 (My book/books)。
 - 我爱她 / 她爱我 (主格宾格形式相同, “我” 和 “她” 形态不变)。

2. 词汇和结构歧义 (Ambiguities):

- 词法歧义 (Lexical): 他很好吃 (He is delicious / He loves to eat)。
- 结构歧义 (Structural):
 - 交叉歧义 (Overlapping): 美国会... (美国/会... vs. 美/国会...)
 - 组合歧义 (Combinatorial): 学生会... (学生/会... vs. 学生会...)

第四部分: 句法分析 (Phrase Grammar and Parsing)

1. **句法 (Syntax)**: 研究句子结构、词序和短语构成的规则。
2. **短语 (Phrase)**: 词语构成的语法单位, 如名词短语(NP)、动词短语(VP)。
3. **两种核心语法理论 (Grammar Theories) - 【高频对比考点】**
 - **短语结构语法 (Phrase Structure Grammar / Constituency Grammar)**:
 - **核心思想**: 句子由短语 (成分) **层级构成**。
 - **产出**: **句法树 (Parse Tree / Syntactic Tree)**。
 - **特点**: 树中包含非终结符节点 (如 NP, VP), 一个词可能对应多个节点 (词本身、词性、短语)。
 - **依存语法 (Dependency Grammar)**:
 - **核心思想**: 句子中的词之间存在直接的**依存关系 (dependencies)**。
 - **关系**: 每个关系连接一个**核心词 (governor)** 和一个**修饰词 (dependent/subordinate)**。
 - **特点**: 结构更扁平, 直接表示词与词的关系, 每个词对应一个节点。
4. **短语结构 vs. 依存语法对比**:

特性	短语结构语法 (Phrase Structure)	依存语法 (Dependency)
词关系	短语构成关系 (phrasal constitution)	核心 - 修饰关系 (head-dependent)
类别	句法功能 (syntactic functional)	句法结构 (syntactic structural)
新节点	是, 一个词对应多个节点	否, 一个词对应一个节点
操作方式	等待短语完整 (waiting for complete phrase)	逐词处理 (word-at-a-time)

1. **句法分析中的歧义 (Parsing Ambiguity)**:
 - **介词短语挂靠 (PP Attachment)**: 最经典的歧义问题。
 - 例: "I shot an elephant **in my pajamas**."
 - 歧义 1: 我 **在我的睡衣里** 射杀了一头大象。(不可能)
 - 歧义 2: 我射杀了一头 **穿着我睡衣的** 大象。(滑稽)
 - 正确理解: 我 **穿着我的睡衣** 射杀了一头大象。

第五部分: 篇章分析 (Discourse Processing)

1. **篇章 (Discourse)**: 超越单个句子的文本单元。
2. **连贯 (Coherence)**: 使篇章成为一个有意义整体的属性。
3. **篇章关系 (Discourse Relations)**: 连接句子或子句的逻辑关系。
 - 例: **解释 (Explanation)**, **阐述 (Elaboration)**, **结果 (Result)**, **并列 (Parallel)**。
4. **篇章分析理论 (Discourse Parsing Theories)**:
 - **修辞结构理论 (RST - Rhetorical Structure Theory)**:
 - 将篇章构建成一个**树状结构 (Tree Structure)**, 表示不同文本单元间的修辞关系。
 - **宾州篇章树库 (PDTB - Penn Discourse Treebank)**:
 - 采用更扁平的结构 (**Flat Structure**), 关注由连接词 (显式或隐式) 连接的论元。
 - **显式 (Explicit)**: "...because Congress hasn't..." (连接词 because)

- **隐式 (Implicit):** "Some have raised their cash positions...
[Implicit=because] High cash positions help..."

第六部分：语料库语言学 (Corpus Linguistics)

1. **语料库 (Corpus):**
 - **定义:** 大规模、结构化的**真实世界**文本集合。
 - **用途:** 进行统计分析、验证语言学规则。
 - **类型:**
 - **单语 (Monolingual) / 多语 (Multilingual)**
 - **对齐平行语料库 (Aligned Parallel Corpora):** 用于机器翻译，句子一一对应。
 - **树库 (Treebank):** 经过完全句法标注的语料库。
2. **著名语料库:**
 - **布朗语料库 (Brown Corpus):** 第一个现代计算机可读的通用语料库。
 - **伦敦-隆德口语语料库 (London-Lund Corpus):** 口语语料。
3. **语料库语言学 (Corpus Linguistics):**
 - **定义:** 基于语料库研究语言的方法。
 - **3A 视角 (3A Perspective):**
 - **标注 (Annotation):** 对文本进行语言学信息的标注 (如词性、句法)。
 - **抽象 (Abstraction):** 将标注映射到理论模型或数据集中。
 - **分析 (Analysis):** 对数据集进行统计探索、验证和泛化。

Lecture 3: Machine Learning Foundations - 核心知识点总结

第一部分：机器学习概览 (Machine Learning Overview)

1. **定义与目标:**
 - **定义:** 机器学习 (ML) 是研究计算机算法如何通过**经验 (experience)** 和**数据 (data)** 自动提升性能的学科。
 - **核心任务:** 解决一个**预测问题**，即给定输入 x ，预测一个“合适的”输出 y 。
2. **ML 任务类型:**
 - **分类 (Classification):** 输出是离散的类别。例：垃圾邮件检测 (Spam / Not Spam)。
 - **回归 (Regression):** 输出是连续的数值。例：电影评分预测 (1~5 分)。
 - **序列标注 (Labeling Problem):** 为输入序列中的每个元素分配一个标签。例：命名实体识别 (NER)、词性标注 (POS)。
3. **机器学习算法三要素: 【高频考点】**
 - **表示 (Representation):** 如何建模 x 和 y 之间的关系。即**假设空间 (Hypothesis Space)**，是所有可能的模型 (如决策函数 $g(x; \theta)$ 或条件概率 $p(y|x; \theta)$) 的集合。
 - **评估 (Evaluation):** 如何评价一个模型的好坏。通常使用**损失函数 (Loss Function)** 和**经验风险最小化 (Empirical Risk Minimization)** 原则。
 - **优化 (Optimization):** 如何在假设空间中找到最优模型参数 θ 。常用的算法是**梯度下降 (Gradient Descent)**。
4. **学习类型:**

- **监督学习 (Supervised Learning):** 最核心, 从有标签的训练数据 (x, y) 中学习。本章重点。
- **无监督学习 (Unsupervised Learning):** 从无标签的数据 x 中学习, 发现数据内在结构。
- **强化学习 (Reinforcement Learning):** 通过与环境的交互和反馈 (奖励/惩罚) 来学习。

第二部分: 线性分类器 (Linear Classifier)

1. 流程:

- **特征提取 (Feature Extraction):** 将原始输入 (如文本) 转换为一个 n 维特征向量 x 。这是一个将非结构化数据结构化的过程。
- **分类:** 在 n 维空间中, 找到一个超平面 (Hyperplane) 作为决策边界 (Decision Boundary), 将不同类别的点分开。

2. 线性判别函数 (Linear Discriminant Function):

- **公式:** $f(x, w) = w^T * x + b$
- **参数:** w 是权重向量 (weights), b 是偏置项 (bias term)。
- **决策:**
 - $f(x, w) > 0 \rightarrow$ 类别 1
 - $f(x, w) < 0 \rightarrow$ 类别 2
 - $w^T * x + b = 0$ 就是决策边界。

第三部分: 逻辑回归 (Logistic Regression - LR)

1. 核心思想:

- 虽然叫“回归”, 但 LR 是一个分类算法。
- 它通过 **Sigmoid (Logistic) 函数** 将线性判别函数的输出 $w^T * x + b$ (范围是实数 R) 映射到 $(0, 1)$ 区间, 使其代表概率。
- **Sigmoid 函数:** $\sigma(z) = 1 / (1 + \exp(-z))$

2. 条件概率:

- $p(y=1|x) = \sigma(w^T * x + b)$
- $p(y=0|x) = 1 - \sigma(w^T * x + b)$

3. 学习过程:

- **损失函数:** 交叉熵损失函数 (Cross-entropy Loss)。其目标是最大化正确标签的对数似然。
- **优化算法:** 梯度下降 (Gradient Descent), 沿着损失函数的负梯度方向更新参数 w 和 b 。
- **过拟合 (Overfitting):** 模型在训练集上表现完美, 但在测试集上表现差。
- **正则化 (Regularization):** 解决过拟合的方法, 在损失函数中加入惩罚项来限制权重的大小。
 - **L1 正则化 (Lasso):** $R(\theta) = \|\theta\|_1$, 倾向于产生稀疏权重 (很多权重为 0), 可用于特征选择。
 - **L2 正则化 (Ridge):** $R(\theta) = \|\theta\|_2^2$, 倾向于产生值较小的权重, 使模型更平滑。

第四部分: 支持向量机 (Support Vector Machine - SVM)

1. 核心思想:

- 在众多可以划分两类样本的超平面中，找到**最好的**一个。
 - **最好的**的定义是：**间隔最大化 (Maximum Margin)**，即决策边界离两侧最近的样本点（支持向量）的距离最大。
2. **关键概念**:
- **间隔 (Margin)**: 决策边界与支持向量之间的距离。
 - **支持向量 (Support Vectors)**: 离决策边界最近的那些点，它们“支撑”着决策边界。
 - **优化目标**: 最大化 $1/\|w\|$ ，等价于最小化 $(1/2) * \|w\|^2$ 。
 - **求解**: 这是一个凸二次规划 (**Convex Quadratic Programming**) 问题，通常通过**拉格朗日乘子法 (Lagrange Multiplier)** 求解其对偶问题 (**Dual Form**)。
3. **核函数 (Kernel Function): 【SVM 核心与高频考点】**
- **动机**: 解决线性不可分问题。
 - **方法**: 将原始特征空间中的样本通过一个映射 $\phi(x)$ 投射到**更高维**的空间，使其在新空间中变得线性可分。
 - **核技巧 (Kernel Trick)**: 在对偶问题中，我们只需要计算 $\phi(x_i)^T * \phi(x_j)$ 的内积，而不需要显式地计算 $\phi(x)$ 。核函数 $K(x_i, x_j) = \phi(x_i)^T * \phi(x_j)$ 可以直接计算这个内积，大大降低了计算复杂度。
 - **常用核函数**:
 - **线性核 (Linear)**: $K(x, z) = x^T * z$
 - **多项式核 (Polynomial)**: $K(x, z) = (x^T * z + 1)^d$
 - **高斯核/径向基函数核 (Gaussian/RBF)**: $K(x, z) = \exp(-\|x-z\|^2 / (2\sigma^2))$

第五部分：结构化学习 (Structured Learning)

1. **定义**:
- 与传统分类器输出单个标签不同，结构化学习的输出是**结构化对象**，如**序列 (sequences)**、**树 (trees)**、**图 (graphs)**。
 - **核心挑战**: 输出空间巨大，无法像简单分类那样穷举。需要考虑输出内部的依赖关系。
 - **典型任务**: 词性标注 (POS)，输入是单词序列，输出是词性序列。

第六部分：隐马尔可夫模型 (Hidden Markov Model - HMM)

1. **模型定义**:
- 一个用于描述含有**隐藏未知参数**的马尔可夫过程的**生成模型 (Generative Model)**。
 - **观测序列 (Observation) x** : 可见的，如单词、拼音。
 - **状态序列 (State) z** : 隐藏的，如词性、汉字。
 - **HMM 三元组**: $\lambda = (A, B, \pi)$
 - **A: 状态转移概率矩阵 (State transition matrix)** - 状态之间的转移概率。 $P(z_{t+1}|z_t)$
 - **B: 观测概率矩阵 (Observation probability matrix)** - 在某个状态下生成某个观测的概率。也叫发射概率 (Emission)。 $P(x_t|z_t)$
 - **π : 初始状态概率向量 (Initial state probabilities)** - 序列开始时处于各个状态的概率。 $P(z_1)$

2. 两个基本假设: 【HMM 核心与高频考点】
 - 齐次马尔可夫假设 (Homogeneous Markov Property): 当前状态 z_t 只依赖于前一个状态 z_{t-1} 。
 - 观测独立假设 (Observational Independence Hypothesis): 当前观测 x_t 只依赖于当前状态 z_t 。
3. 三个基本问题: 【HMM 核心与高频考点】
 - 评估 (Evaluation): $P(x|\lambda)$ - 给定模型 λ 和观测序列 x , 计算该序列出现的概率。
 - 算法: 前向算法 (Forward Algorithm) / 后向算法 (Backward Algorithm)。利用动态规划思想, 避免暴力计算。
 - 解码 (Decoding): $\operatorname{argmax}_z P(z|x, \lambda)$ - 给定模型 λ 和观测序列 x , 找到最有可能的隐藏状态序列 z 。
 - 算法: 维特比算法 (Viterbi Algorithm)。同样是动态规划, 找到全局最优路径。
 - 学习 (Learning): $\operatorname{argmax}_\lambda P(x|\lambda)$ - 给定观测序列 x , 如何调整模型参数 λ 使该序列出现的概率最大。
 - 算法: Baum-Welch 算法 (前向-后向算法), 是 EM (Expectation-Maximization) 算法的一个特例。

第七部分: 条件随机场 (Conditional Random Field - CRF)

1. 模型定义:
 - 是一个判别模型 (Discriminative Model), 直接对条件概率 $P(z|x)$ 进行建模。
 - 是概率无向图模型 (Undirected Graph Model)。
 - 线性链 CRF (Linear-chain CRF) 是最常见的形式, 用于序列标注。
2. HMM vs. CRF 对比: 【高频对比考点】

特性	HMM (隐马尔可夫模型)	CRF (条件随机场)
模型类型	生成模型 (Generative), 对联合概率 $P(x, z)$ 建模。	判别模型 (Discriminative), 对条件概率 $P(z x)$ 建模。
假设	严格的观测独立假设。	无严格假设, 可以定义任意复杂的特征。
特征	观测只依赖当前状态。	特征函数可以利用整个观测序列 x 的信息。
归一化	局部归一化 (Local Normalization)。	全局归一化 (Global Normalization), 计算复杂。
速度	训练和预测都非常快。	训练慢, 预测快 (类似 HMM 的 Viterbi)。

- **核心优势:** CRF 摆脱了 HMM 的严格独立性假设, 可以利用更丰富的、全局的特征, 因此在很多序列标注任务上表现通常优于 HMM。

Lecture 4: Deep Learning Foundation - 核心知识点总结

第一部分：从机器学习到深度学习 (From ML to DL)

1. 传统机器学习 (ML) 的局限:
 - 流程: Text -> Feature Engineering -> ML Model -> Output
 - 核心: 特征工程 (Feature Engineering) 至关重要, 且耗费大量人力。模型性能的上限很大程度上取决于特征的好坏。
 - 特征工程步骤: 预处理、特征创建、特征变换 (提取/选择)。
2. 深度学习 (DL) 的革命:
 - 流程: Text -> Representation Learning -> Output
 - 核心: 将“特征工程”自动化, 模型可以端到端 (end-to-end) 地从原始数据中自动学习分层次的特征表示 (Representation Learning)。
 - 原理: 通过堆叠多个简单的非线性函数 (层), 形成深度网络, 从而获得强大的表达能力 (expression ability)。 $y = f_n(\dots f_2(f_1(x))\dots)$
3. DL 兴起的原因:
 - 大数据 (Large amounts of data): 提供了足够的训练样本。
 - 硬件发展 (Cheaper and faster computers): GPU 提供了强大的并行计算能力。
 - 技术进步 (Advanced ML techniques): 更好的网络结构、激活函数、优化算法等。
4. DL 三要素 (同 ML):
 - 表示 (Representation): 神经网络 (Neural Network)。
 - 评估 (Evaluation): 交叉熵损失 (Cross-entropy loss) 等。
 - 优化 (Optimization): 随机梯度下降 (Stochastic Gradient Descent - SGD) 及其变种 (Adam, etc.)。

第二部分：人工神经网络基础 (Neuron and ANN)

1. 人工神经元 (Artificial Neuron):
 - 模型: $\text{output} = \text{activation_function}(w^T \cdot x + b)$
 - 组成:
 - 输入 (Inputs) x
 - 权重 (Weights) w
 - 偏置 (Bias) b
 - 激活函数 (Activation function): 引入非线性, 如 Sigmoid, Tanh, ReLU。
2. 人工神经网络 (ANN):
 - 由多个神经元按一定拓扑结构连接而成。
 - 基本类型: 前馈网络 (Feed Forward), 记忆网络 (Memory), 图网络 (Graph)。
 - 前馈神经网络 (Feed Forward Neural Network, FFNN): 信息单向流动, 不形成环路。
 - 万能近似定理 (Universal Approximation Theorem): 【核心理论】 一个包含单隐藏层的、有限数量神经元的前馈网络, 可以以任意精度近似任何连续函数。这证明了神经网络强大的表达能力。

第三部分：循环神经网络 (Recurrent Neural Network - RNN) - 【序列建模核心】

1. 核心理想:
 - “循环”: 在序列的每个时间步, 模型重复使用相同的权重 W 。

- **记忆:** 每个时间步的输出 h_t (隐藏状态) 不仅依赖于当前输入 x_t , 还依赖于上一个时间步的隐藏状态 h_{t-1} 。
 - **公式:** $h_t = f(W * h_{t-1} + U * x_t + b)$
 - **应用:** 语言模型、序列标注 (POS tagging)、文本分类、文本生成。
2. **优势:**
- 能处理任意长度的输入序列。
 - 模型大小与序列长度无关 (参数共享)。
 - 理论上能捕捉长距离依赖。
3. **劣势 - 长距离依赖问题 (Long-Term Dependencies Problem):**
- **梯度消失/爆炸 (Vanishing/Exploding Gradients):** 在反向传播时, 梯度经过多次连乘后会变得极小 (消失) 或极大 (爆炸), 导致模型难以学习到序列远端的依赖关系。

第四部分: LSTM 与 GRU - RNN 的变种

1. **长短期记忆网络 (Long Short-Term Memory - LSTM): 【RNN 考点核心】**
- **动机:** 解决 RNN 的长距离依赖问题。
 - **核心结构:** 引入细胞状态 (Cell State, c_t) 和三个门 (Gates)。
 - **细胞状态 c_t :** 像传送带一样, 负责传递长期信息, 信息在上面流动受到的影响很小。
 - **遗忘门 (Forget Gate):** 决定从上一个细胞状态 c_{t-1} 中丢弃哪些信息。
 - **输入门 (Input Gate):** 决定将当前输入产生的哪些新信息存入细胞状态 c_t 。
 - **输出门 (Output Gate):** 决定从细胞状态 c_t 中输出哪些信息到隐藏状态 h_t 。
 - **本质:** 通过门控机制, 有选择地让信息通过, 从而更好地保存和利用长期信息。
2. **门控循环单元 (Gated Recurrent Unit - GRU):**
- **动机:** LSTM 的简化版, 参数更少, 计算效率更高。
 - **核心结构:** 将遗忘门和输入门合并为更新门 (Update Gate), 并引入重置门 (Reset Gate)。没有独立的细胞状态。
3. **LSTM vs. GRU:**
- 两者都是解决长距离依赖的有效方法。
 - **经验法则:** 数据量大、依赖关系特别长时, LSTM 可能是更好的默认选择。追求速度和更少参数时, 可以尝试 GRU。
4. **双向 RNN (Bidirectional RNNs):**
- **思想:** 同时使用一个前向 RNN 和一个后向 RNN, 并将它们的隐藏状态在每个时间步拼接起来。
 - **优势:** 在 t 时刻的表示, 能同时编码过去 (past) 和未来 (future) 的上下文信息。
5. **多层 RNN (Multi-layer RNNs):**
- 将 RNN 堆叠起来, 上一层的输出作为下一层的输入。
 - **优势:** 能够学习到更高层次、更抽象的特征表示。

第五部分: 卷积神经网络 (Convolutional Neural Network - CNN)

1. **核心思想 (NLP 中):**

- **卷积核 (Filter/Kernel)**: 一个小的矩阵, 在输入文本的词向量矩阵上滑动, 进行卷积操作, 以**提取局部特征 (n-gram)**。
- **参数共享**: 同一个卷积核在整个文本上共享参数, 大大减少了模型参数量。
- **多卷积核**: 使用不同尺寸的卷积核可以同时捕捉不同大小的 n-gram 特征 (如 bi-gram, tri-gram)。
- **池化 (Pooling)**: 通常是**最大池化 (Max-pooling)**, 从卷积操作得到的特征图中提取最重要的特征 (最大值), 实现降维和特征聚合。
- **典型模型: TextCNN**。

2. 为什么 TextCNN 有效:

- 对于很多文本分类任务 (如情感分析), 关键信息往往由少数几个词或短语决定 (如 "not bad", "quite good"), CNN 擅长捕捉这种**局部关键模式**。
- **不擅长**: 捕捉长距离依赖关系, 因此不适用于语言模型等任务。

第六部分: 序列到序列模型与注意力机制 (Seq2Seq & Attention)

1. 编码器-解码器 (Encoder-Decoder / Seq2Seq):

- **结构**: 由一个 **Encoder** 和一个 **Decoder** 组成, 通常都是 RNN。
- **流程**:
 1. **Encoder**: 读取整个输入序列, 将其压缩成一个固定长度的上下文向量 (**Context Vector, C**)。
 2. **Decoder**: 以 C 为初始状态, 逐步生成输出序列。
- **瓶颈: 信息瓶颈 (Information Bottleneck)**, 所有输入信息都必须被压缩到一个固定长度的向量 C 中, 对于长序列会丢失信息。

2. 注意力机制 (Attention Mechanism): 【现代 NLP 模型核心中的核心】

- **动机**: 解决 Seq2Seq 的信息瓶颈问题。
- **核心思想**: 让 Decoder 在生成每个词时, 不再依赖单一的上下文向量 C, 而是可以“**关注**”到 Encoder 所有隐藏状态中的**相关部分**, 并给它们分配不同的**权重**。
- **流程 (K-Q-V)**:
 1. **Query (Q)**: 当前 Decoder 的状态, 表示“我想找什么信息”。
 2. **Keys (K)**: Encoder 的所有隐藏状态, 表示“我这里有这些信息”。
 3. **计算注意力分数**: $\text{score} = \text{score_function}(Q, K)$, 得到每个 Key 的重要性。
 4. **归一化 (Softmax)**: 将分数转换为概率分布 (注意力权重 α)。
 5. **加权求和**: $\text{context_vector} = \sum(\alpha_i * V_i)$, 其中 Values (V) 通常就是 Keys。
- **优势**: 允许模型直接建立输入和输出序列任意部分之间的联系, 极大地提升了长序列任务的性能。

第七部分: Transformer

1. 核心思想: "Attention Is All You Need"

- 完全抛弃 RNN 和 CNN 的**循环和卷积结构**, 只使用**注意力机制**来构建模型。

2. 核心组件:

- **自注意力 (Self-Attention)**: 在**同一个序列内部**计算注意力, 让序列中的每个词都能关注到序列中所有其他的词, 从而捕捉序列内部的依赖关系。
- **多头注意力 (Multi-head Attention)**: 将自注意力过程重复多次 (多个“头”), 每个头学习不同的注意力模式 (如关注句法、关注距离等), 然后将结果拼

接。这使得模型能从不同的“子空间”中学习信息。

- **位置编码 (Positional Encoding)**: 因为自注意力本身不包含位置信息, 需要显式地将位置信息加入到输入 embedding 中。
- **残差连接 (Residual Connection) & 层归一化 (Layer Norm)**: 帮助缓解梯度消失问题, 加速模型训练和稳定。
- **前馈网络 (Feed Forward Network)**: 在每个自注意力层后接一个简单的 FFN, 增加模型的非线性能力。

3. 优势:

- **并行计算**: 彻底摆脱了 RNN 的序列依赖, 可以实现高度并行化, 训练速度远超 RNN。
- **全局依赖捕捉**: 自注意力直接连接任意两个位置, 捕捉长距离依赖的能力更强。
- 成为 BERT、GPT 等现代大规模预训练语言模型的基础。

Lecture 5: Language Model - 核心知识点总结

第一部分: 语言模型基础 (Language Modeling Basics)

1. 定义 (Definition):

- **核心任务**: 预测序列中的下一个词是什么, 即计算条件概率 $P(w_{t+1} | w_1, \dots, w_t)$ 。
- **广义任务**: 为一个完整的词序列 (句子) 分配一个概率 $P(w_1, w_2, \dots, w_T)$ 。一个好的语言模型会给符合语法、语义流畅的句子赋予更高的概率。
- **计算方法**: 使用**概率链式法则 (Chain Rule of Probability)** 将句子概率分解为一系列条件概率的乘积。

2. 应用 (Applications):

- **语音识别**: 区分同音词, 如 "I went to a party" vs. "Eye went two a bar tea"。
- **机器翻译**: 从多个候选翻译中选择最通顺、最符合目标语言习惯的句子。
- **拼写纠错**: "Their are problems" -> "There are problems"。
- **中文分词**: "写得太平淡了" vs. "写得太/平淡/了"。
- **文本生成、问答、摘要**等几乎所有 NLP 任务。

3. 评估指标 (Evaluation):

- **困惑度 (Perplexity, PP)**: 最重要的评估指标。
 - **定义**: 测试集上每个词的平均不确定性的度量, 是测试集概率的倒数再开 N 次方根。
 - **直观理解**: 模型在预测下一个词时, 平均有多少个“候选词”。
 - **评判标准**: 困惑度越低, 语言模型越好。
 - **关系**: 最小化困惑度等价于最大化测试集句子的概率。

第二部分: 统计语言模型 (Statistical Language Model - SLM)

1. N-gram 模型: 【SLM 核心与高频考点】

- **动机**: 概率链式法则中的完整历史 $(w_1, \dots, w_{t-1})$ 太长, 导致数据极度稀疏, 无法估计。
- **马尔可夫假设 (Markov Assumption)**: 简化历史, 假设当前词的概率只依赖于它前面的 n-1 个词。

- 常见类型:
 - **Unigram (n=1):** $P(w_t)$ (上下文无关)
 - **Bigram (n=2):** $P(w_t | w_{t-1})$
 - **Trigram (n=3):** $P(w_t | w_{t-2}, w_{t-1})$
 - 概率估计: **最大似然估计 (Maximum Likelihood Estimation - MLE)**, 即通过“数数”来计算。
 - $P(w_i | w_{1:i-1}) = \text{Count}(w_{1:i-1}, w_i) / \text{Count}(w_{1:i-1})$
 - **N-gram 的局限:**
 - **数据稀疏性 (Sparsity):** 随着 N 的增大, 参数空间呈指数级增长 (V^N), 大部分 n-gram 在训练集中从未出现。
 - **长距离依赖 (Long Distance Dependencies):** 无法捕捉超出 n-gram 窗口的依赖关系 (如主谓一致)。
2. 平滑技术 (Smoothing): **【解决数据稀疏性的关键】**
- **问题 (Zeros):** 如果一个 n-gram 在训练集中没出现, 其概率为 0, 会导致整个句子的概率也为 0。
 - **核心思想:** 从“见过”的事件中“偷”一部分概率, 分配给“没见过”的事件, 保证任何序列的概率都不会是 0。
 - 常见方法:
 - **拉普拉斯平滑 / 加一平滑 (Laplace / Add-One Smoothing):**
 - 方法: 给所有 n-gram 的计数都加 1。
 - 缺点: 对于大词汇表, 给未见事件分配了过多的概率, 效果不佳。
 - **加 K 平滑 (Add-K Smoothing):** 将加 1 改为加一个小的分数 k, 效果稍好。
 - **回退 (Back-off):** 如果高阶模型 (如 trigram) 的数据不足 (计数为 0), 则“回退”到低阶模型 (如 bigram)。
 - **插值 (Interpolation):** 效果通常比回退好。将不同阶的 n-gram 模型 (如 trigram, bigram, unigram) 加权平均起来。权重 λ 可以在留存集 (held-out data) 上学习。

第三部分: 神经网络语言模型 (Neural Language Model - NLM)

1. 统计语言模型 (SLM) 的根本缺陷:
 - **数据稀疏:** 核心问题。
 - **无法捕捉语义相似性:** N-gram 将词视为离散符号 (One-hot), 无法知道 "cat" 和 "kitty" 是相似的。如果训练集中有 "a cute cat", 但没有 "a cute kitty", SLM 无法泛化。
2. 词表示的演进:
 - **One-hot Representation:** 高维、离散、稀疏。词与词之间相互正交, 无法表示相似度。
 - **词袋模型 (Bag-of-words):** 忽略词序, 只统计词频。
 - **TF-IDF:** 衡量一个词在文档中的重要性。TF (词频) * IDF (逆文档频率)。
 - **分布式表示 / 词嵌入 (Distributed Representation / Word Embedding):**
 - **核心思想:** 用一个低维、稠密、连续的实数向量来表示一个词。
 - **优势:** 向量空间中的距离可以表示语义相似度 (e.g., $\text{vector}(\text{'king'}) - \text{vector}(\text{'man'}) + \text{vector}(\text{'woman'}) \approx \text{vector}(\text{'queen'})$)。

3. 前馈神经网络语言模型 (NNLM - Bengio, 2003):

- **结构:** Input ($n-1$ 个词的 embedding) $\rightarrow$ Hidden Layer $\rightarrow$ Output Layer (Softmax)
- **优势:** 通过词嵌入解决了语义相似性问题, 相似的上下文会得到相似表示。
- **劣势:**
 - 仍然受限于固定窗口大小 ($n-1$)。
 - 计算速度慢。

4. 循环神经网络语言模型 (RNNLM):

- 使用 RNN (如 LSTM) 来建模语言模型。
- **优势:**
 - 能处理任意长度的上下文历史。
 - 自然地解决了固定窗口问题。
 - 成为后续更强大模型的基础。

第四部分: 预训练语言模型 (Pre-trained Language Model - PLM)

1. 静态词嵌入 (Static Word Embeddings):

- **代表:** Word2vec, GloVe, FastText。
- **Word2vec:**
 - **思想:** “一个词的含义由其上下文决定”。
 - **模型:**
 - **Skip-gram:** 给定中心词, 预测上下文词。
 - **CBOW:** 给定上下文词, 预测中心词。
 - **优化:** 负采样 (Negative Sampling), 将多分类问题转为二分类问题, 大大提高效率。
- **问题:** 一词多义 (Polysemy)。一个词 (如“bank”) 只有一个固定的向量, 无法根据上下文区分其不同含义 (银行 vs. 河岸)。

2. 上下文相关的词表示 (Contextual Representations): 【现代 NLP 范式核心】

- **核心思想:** 一个词的表示应该是动态的, 随其上下文的变化而变化。
- “预训练-微调” 范式 (Pre-training and Fine-tuning):

1. **预训练 (Pre-training):** 在海量无标签文本上训练一个通用的语言模型。

2. **微调 (Fine-tuning):** 在特定的下游任务 (如分类、问答) 的少量有标签数据上, 对预训练好的模型进行参数微调。

3. 代表模型发展史:

- **ELMo (2018):**
 - 使用双向 LSTM (biLM) 进行预训练。
 - 将预训练好的模型各层的表示加权求和作为下游任务的特征, 模型本身不参与微调 (Feature-based)。
- **GPT (Generative Pre-Training, OpenAI, 2018):**
 - 使用 Transformer Decoder 结构。
 - 是单向 (Unidirectional) 语言模型。
 - 开启了微调 (Fine-tuning) 的范式, 整个模型参数在下游任务中都会更新。
- **BERT (Bidirectional Encoder Representations from Transformers, Google, 2018):**
 - 使用 Transformer Encoder 结构。

- **核心创新:**
 - **掩码语言模型 (Masked Language Model, MLM):** 随机 mask 掉输入中的一些词, 然后预测这些被 mask 的词。这使得模型可以同时看到左右两边的上下文, 是真正的双向 (Bidirectional) 模型。
 - **下一句预测 (Next Sentence Prediction, NSP):** 预训练任务, 判断两个句子是否是连续的。
- **后 BERT 时代:** RoBERTa, XLNet, ALBERT, T5, ERNIE... 都是对 BERT 预训练任务、模型结构或训练策略的改进。
- **大模型时代:** GPT-3, PanGu, WuDao 等, 通过巨大的模型规模和数据量, 展现出强大的上下文学习 (In-Context Learning) / 少样本学习 (Few-shot Learning) 能力。
- 4. **模型压缩 (Model Compression):**
 - **动机:** 大模型参数多, 推理慢, 部署困难。
 - **方法:** 剪枝 (Pruning), **知识蒸馏 (Distillation)**, 量化 (Quantization)。

Lecture 6: Word Processing - 核心知识点总结

第一部分: 中文分词 (Chinese Word Segmentation - CWS)

1. **为什么需要分词:**
 - 中文书写时词与词之间没有天然的分隔符 (如空格)。
 - **分词是大多数中文 NLP 任务的基础**, 如信息检索、机器翻译、句法分析等。分词的准确性直接影响下游任务的效果。
 - **错误分词导致语义错误:**
 - 和服/务 vs. 和/服务
 - 查/金泰 (人名) vs. 查金/泰 (姓查名金泰)
 2. **分词的主要挑战 (Two Main Challenges):**
 - **歧义 (Ambiguity):** 一段文本有多种合法的切分方式。
 - **交叉型歧义 (Overlapping):** 人才|能 vs. 人|才能
 - **组合型歧义 (Combinational):** 个人 vs. 个|人
 - **混合型歧义 (Mixed):** 太平|淡 vs. 太|平淡
 - **未登录词 / OOV (Unknown / Out-of-Vocabulary words):** 词典中没有收录的词, 如人名、地名、机构名、网络新词等。
 - **OOV 是分词错误的主要来源**, 其导致的错误率远高于歧义。
 3. **分词规范与评测 (Standards and Benchmarks):**
 - **规范:** GBT 13715-1992, 台湾中研院规范, 人民日报语料库规范等。
 - **评测集:** SIGHAN Bakeoff 系列评测提供了标准的语料库 (PKU, MSR, AS, CityU) 和评测方法。
 - **开放测试 (Open Track):** 可以使用额外资源 (如词典)。
 - **封闭测试 (Close Track):** 只能使用官方提供的训练数据。
 - **评测指标:** Precision (精确率), Recall (召回率), F-measure (F1 值), 以及对 OOV 的专门召回率。
-

第二部分：分词算法 (Segmentation Algorithms)

1. **核心思想**: 将文本与一个预先构建好的词典进行匹配。
2. **主要算法**:
 - **正向最大匹配 (Forward Maximum Matching, FMM)**:
 - 从左到右, 贪心地匹配词典中最长的词。
 - **逆向最大匹配 (Backward Maximum Matching, BMM)**:
 - 从右到左, 贪心地匹配词典中最长的词。
 - **通常比 FMM 更准确**, 因为中文的中心语常在后面。
 - **双向最大匹配 (Bi-directional Maximum Matching)**:
 - 同时执行 FMM 和 BMM, 当结果不一致时, 通过一些启发式规则 (如比较分词数) 来选择更优的结果。
3. **缺点**:
 - **严重依赖词典质量**, 无法处理 OOV。
 - 难以解决**组合型歧义**。
1. **核心思想**: 利用统计信息 (如互信息、t-score、EM 算法) 判断相邻字/词的粘合程度。
2. **词格 (Word Lattice) 方法**:
 - **流程**:
 1. 根据词典找出句子中所有可能的词, 构成一个**有向无环图 (DAG)**, 即词格。
 2. 将分词问题转化为在词格中寻找**最可能路径**的问题。
 - **路径搜索**:
 - 可以使用**机器学习算法** (如最大概率路径、最短路径) 来寻找最佳切分。
 - 典型算法: 维特比算法 (Viterbi)。
1. **核心思想**: 将分词问题转化为一个**字符级别的序列标注 (Character Labeling)** 问题。
2. **标注体系 (Tagging Scheme): 【高频考点】**
 - 为每个字符打上一个标签, 表示它在一个词中的位置。
 - **BMES 体系**:
 - B: Begin (词的开始)
 - M: Middle (词的中间)
 - E: End (词的结尾)
 - S: Single (单字成词)
 - **BIO 体系**: B: Begin, I: Inside (等同于 M 和 E), O: Outside (非词部分, 但在分词中不常用)。
3. **代表模型**:
 - **条件随机场 (CRF)**: 传统机器学习时代的 SOTA (State-of-the-art) 模型, 能够利用丰富的上下文特征, 效果优于 HMM。
 - **神经网络方法 (Neural-based)**:
 - **模型结构**: Embedding -> BiLSTM -> CRF 是一个非常经典的结构。
 - **Embedding 层**: 将字符映射为低维向量。
 - **BiLSTM 层**: 捕捉长距离的上下文依赖关系。
 - **CRF 层**: 在模型顶部加入, 学习标签之间的约束关系 (如 B 后面不能是 S), 保证输出的标签序列是合法的。

- **发展:** 后续引入更复杂的结构如 Attention、预训练模型 (BERT) 等, 进一步提升性能。
-

第三部分: 未登录词识别 (Unknown Word Identification / OOV)

1. OOV 主要类型:

- **命名实体 (Named Entities):** 人名、地名、机构名。
- **缩略语 (Abbreviation):** 国考 -> 国家公务员考试
- **派生词 (Derived words):** 审计 -> 审计员
- **复合词 (Compounds):** 光+敏感 -> 光敏感
- **网络新词:** 给力, 酱油帝

2. 识别方法:

- **规则法 (Rule-based):** 利用构词规则、拒绝规则等。
- **统计法 (Statistics-based):**
 - **互信息 (Mutual Information):** 衡量字与字之间的粘合度。
 - **左右熵 (Left/Right Entropy):** 衡量一个字符串左右邻字的丰富度。
如果一个字符串是独立的词, 其左右邻字通常比较丰富, 熵值较高。
 - **词内成词概率 (In-word probability):** 衡量一个字串是否像一个词。
- **机器学习法:** 将 OOV 识别看作一个分类或序列标注问题。

3. 典型 OOV 识别任务 - 人名识别:

- **特点:** 构成相对任意, 但有一定规律。
- **线索:**
 - **姓氏:** 中国常用姓氏数量有限 (<300)。
 - **上下文:** 称谓词 (总理、厂长)、特定动词 (任命、接见)。
 - **偏旁部首 (Radical):** 用于性别、褒贬色彩的判断 (如女字旁、玉字旁)。
- **音译名:** 常用音译字 (尔、姆、斯)、结构特征 (·)。
- **HMM 方法:**
 - 将人名识别转化为序列标注问题, 使用 BME (Begin, Middle, End) 或 BMM...E 标记人名。
 - 通过 Viterbi 算法找到最优标注路径, 从而识别人名。

4. 机构名识别:

- **特点:** 结构复杂, 长度不定, 可能嵌套。
- **线索:**
 - **核心词/后缀:** 公司, 基金会, 大学 等是右边界的强标志。
 - **前缀:** 经常包含地名、专有名。
 - 左边界的确定是难点。

5. 地名识别:

- **特点:** 可利用地名词典, 但新地名、简称等仍是挑战。
- **线索:**
 - **后缀:** 省, 市, 县, 村 等。
 - **上下文规则:** 如两个候选地名由顿号连接, 则另一个也很可能是地名。

6. 缩略语标准化 (Abbreviation Normalization):

- **任务:** 将缩略语还原为其全称。
- **方法:**

- 生成候选全称:
 - 基于映射表: 为缩略语中的每个字生成可能的扩展词 (如 大 -> 大学, 大会)。
 - 基于缩略语词典: 直接查找。
- 消歧 (Disambiguation):
 - 将还原问题看作 **N-gram 解码** 或 **分词** 问题, 利用语言模型在所有候选全称中选择概率最高的那个。

Lecture 7: Word Meaning & Word Sense Disambiguation - 核心知识点总结

第一部分: 词义与语义资源 (Word Meaning & Semantic Resources)

1. 核心概念 (Terminology) - 【高频考点, 务必掌握】

- 词义 (Word Sense): 一个词所具有的多种含义之一。
- 语义学 (Semantics): 研究语言意义的学科。
- 同形同音异义词 (Homonyms): 写法和读音相同, 但意义不相关。例: bank (银行) vs. bank (河岸)。
- 多义词 (Polyseme): 一个词有多个相互关联的意义。例: bank (金融机构 vs. 存钱的建筑 vs. 血库)。
- 同义词 (Synonyms): 意义相同或相近的词。例: student / pupil。
- 同义词集 (Synset): 一个或多个同义词构成的集合, 代表一个唯一的概念。这是 WordNet 的核心组织单位。
- 反义词 (Antonyms): 意义相反的词。例: big / small。
- 上位词 (Hypernym): 更概括的词 (is-a-kind-of)。例: animal 是 dog 的上位词。
- 下位词 (Hyponym): 更具体的词。例: dog 是 animal 的下位词。
- 整体词 (Holonym): 表示整体的词 (is-a-part-of)。例: tree 是 bark (树皮) 的整体词。
- 分体词 (Meronym): 表示部分的词。例: finger 是 hand 的分体词。
- 转喻 (Metonymy): 用一个紧密相关的事物来指代另一个。例: 用 White House 指代 President。
- 行态词 (Troponym): 描述动作方式的词。例: lisp (口齿不清地说) 是 talk 的行态词。
- 蕴含 (Entailment): 一个动词的发生必然导致另一个动词的发生。例: snore (打鼾) 蕴含 sleep (睡觉)。

2. 词义的计算机表示:

- 基于义元 (Sememe): 将词义拆解为不可再分的最小语义单位 (义元)。例: 翻腾 的义元可能包括 {exercise|锻炼}、{roll|滚}。
- 基于同义词集: 用一个同义词集 (synset) 和其释义 (gloss) 来表示一个概念。

3. 重要的语义资源 (Semantic Resources) - 【核心知识库】

- WordNet (英文):
 - 性质: 一个大规模的词汇数据库 (lexical database), 既是词典也是

同义词库。

- **组织方式:** 以**同义词集 (synset)** 为基本单位, 通过丰富的**语义关系** (上下位、整体分体等) 将 synset 连接成一个网络。
- **结构:** 名词和动词被组织成**层次结构 (hierarchies)**。
- **应用:** 词义消歧、词汇相似度计算、知识图谱构建等。
- **词汇相似度计算方法:**
 - **基于路径 (Path-based):** 在层次结构中, 两个概念的路径越短, 相似度越高。缺点是假设所有边权重相同。
 - **基于信息内容 (Information Content - Resnik):** 相似度取决于其**最低共同祖先 (LCS)** 的信息量 (即概率的负对数)。LCS 越具体 (概率越低, 信息量越大), 相似度越高。
- **HowNet (知网, 中文):**
 - **性质:** 一个基于义元 (Sememe) 的常识知识库。
 - **核心思想:** 任何概念都可以由一个封闭的义元集合通过结构化标记语言来定义。
 - **结构:** **图结构 (Graph structure)**, 不同于 WordNet 的树状层次, 能更好地表示跨词性的关系。
 - **应用:** 词汇相似度计算、词表示学习、事件检测等。
- **CILIN (同义词词林, 中文):**
 - **性质:** 当代汉语同义词词典。
 - **结构:** 三级**层次树结构** (大、中、小类), 同一小类下的词共享同一概念。
 - **特点:** 比 WordNet 更细粒度地划分了同义/反义关系。
- **CCD (中文概念词典):**
 - **性质:** 仿照 WordNet 框架构建的中英双语词典, 结构上与 WordNet 兼容。
 - **特点:** 记录了更丰富的词法、句法信息 (如构词法、情感极性), 缓解了 WordNet 跨词性关联不足的问题。

第二部分: 词义消歧 (Word Sense Disambiguation - WSD)

1. **定义与动机:**
 - **任务:** 根据上下文 (context), 自动确定一个多义词在特定语境中的**确切含义 (sense)**。
 - **本质:** 一个**分类问题**, 其中“类别”是词的各个义项, “证据”是词的上下文。
 - **动机:** NLP 的核心挑战之一, 对机器翻译、信息检索、信息抽取等任务至关重要。
2. **基本假设 (Yarowsky, 1995): 【WSD 核心理论】**
 - **一词一搭配 (One Sense Per Collocation):** 附近的词 (搭配) 为目标词的义项提供了强有力的线索。准确率高达 95-96%。
 - **一文一义 (One Sense Per Discourse):** 在一篇文档中, 一个多义词倾向于保持同一个义项。对名词有效, 对动词不一定。
3. **WSD 算法分类:**
 - **核心:** 依赖于词典、义类词典、知识库等外部资源, **不需要大规模标注语料**。
 - **Lesk 算法 (Overlap-based):**

- **思想:** 比较目标词各个义项的**释义 (Sense Bag)** 与其上下文词的**释义 (Context Bag)** 之间的**重叠度**。重叠度最高的义项即为正确义项。
- **基于 Thesaurus 的方法 (Walker's Algorithm):**
 - **思想:** 比较目标词义项的**主题类别**与上下文词的**主题类别**的匹配度。
- **概念密度法 (Conceptual Density):**
 - **思想:** 在 WordNet 等层次结构中, 计算一个义项与上下文中其他词的概念距离。**距离越近, 密度越高**, 该义项越可能是正确的。
- **缺点:** 严重依赖知识库的完备性, 存在稀疏匹配问题 (释义太短、专有名词缺失)。
- ① **监督学习 (Supervised):** 目前性能最好, 但需要大量人工标注的语料。
 - **特征工程:** 提取上下文特征, 如:
 - **搭配特征:** 目标词周围的词及其词性 ($w-1, w+1, w-2, \dots$)。
 - **词袋特征:** 上下文窗口内的所有词。
 - **代表算法:**
 - **朴素贝叶斯 (Naïve Bayes):** 简单有效。
 - **决策列表 (Decision Lists):** 基于“一词一搭配”思想, 将最具有预测能力的特征 (搭配) 排在列表顶端, 按序匹配。
 - **SVM, k-NN:** 也是常用的分类器。
- ② **半监督学习 (Semi-supervised):** 结合少量有标签数据和大量无标签数据。
 - **自举法 / 引导法 (Bootstrapping - Yarowsky's Algorithm):** 【半监督经典】
 1. 从少量**种子样本 (seed examples)** 开始 (人工标注或利用高置信度搭配)。
 2. 用种子样本训练一个初始分类器。
 3. 用该分类器去标注无标签数据。
 4. 将标注结果中**置信度高**的样本加入种子集。
 5. **重复** 2-4 步, 直到收敛。
 - ③ **无监督学习 (Unsupervised):** 不需要人工标注数据, 进行词义归纳 (Word Sense Induction)。
 - **思想:** 基于分布假设 (相似的上下文对应相似的意义), 对词的上下文进行聚类, 每个聚类代表一个潜在的义项。
 - **代表方法:**
 - **基于 Thesaurus 的方法:** 利用词典中不同主题的词汇表作为判别器。
 - **Lin's Approach:** 两个词如果出现在相同的局部语境中, 则它们可能有相似的意义。
 - **混合方法:** 结合知识库和语料库信息。
 - **神经网络方法:**
 - 将 WSD 视为序列标注任务, 使用 **BiLSTM** 等模型。
 - 利用上下文相关的**词嵌入** (如 BERT), 在模型中直接编码了词义信息, 隐式地进行 WSD。
- 4. **WSD 评测边界 (Bounds):**
 - **上限 (Upper Bound):** 人类标注员之间的一致性 (通常在 95%左右)。
 - **下限 / 基线 (Lower Bound / Baseline):** **最频繁义项 (Most Frequent**

Sense, MFS), 即总是选择一个词在语料中出现最多次的那个义项。这是一个非常强的基线。

Lecture 8: Parsing I & II - 核心知识点总结

第一部分：句法分析基础 (Syntactic Analysis Basics)

- 1. 目标与需求:
 - 目标: 确定一个句子的句法结构 (syntactic structure), 判断其是否符合给定的语法规则。
 - 核心任务: 解决语言中的结构歧义 (structural ambiguity)。
 - 两大要素:
 - 语法 (Grammar): 定义允许的结构规则。
 - 解析算法 (Parsing Algorithm): 分析句子并确定其结构的方法。
- 2. 生成语法 (Generative Grammar):
 - 一套可以生成语言中所有合法句子的规则。
 - 形式化定义: $G = (V, T, P, S)$
 - V: 非终结符 (Non-terminals) 集合 (大写字母, e.g., NP, VP)。
 - T: 终结符 (Terminals) 集合 (小写字母/单词, e.g., 'the', 'cat')。
 - P: 产生式规则 (Productions) 集合 (e.g., $S \rightarrow NP VP$)。
 - S: 起始符号 (Start symbol)。

第二部分：乔姆斯基谱系 (Chomsky Hierarchy) - 【高频理论考点】

这是对形式语言进行分类的层次结构，从限制最强到最弱分为四种类型：

类型	语法名称	产生式规则	识别自动机	复杂度
Type-3	正则语法 (Regular)	$A \rightarrow a$ 或 $A \rightarrow aB$ (右线性) / $A \rightarrow Ba$ (左线性)	有限状态自动机 (FSA)	$O(n)$
Type-2	上下文无关语法 (Context-Free, CFG)	$A \rightarrow \alpha$ (左侧为单个非终结符)	下推自动机 (PDA)	$O(n^3)$
Type-1	上下文有关语法 (Context-Sensitive)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ (A 的重写依赖上下文 α, β)	线性有界自动机 (LBA)	PSPACE-complete
Type-0	无限制语法 (Unrestricted)	无限制	图灵机 (Turing Machine)	递归可枚举

核心:

- 自然语言的句法结构主要用 上下文无关语法 (CFG, Type-2) 来建模，因为它在表达能力和计算效率之间取得了很好的平衡。
- CFG 强大到足以处理自然语言的大部分结构，同时又足够受限，可以设计出高效的解析器（如 $O(n^3)$ ）。

第三部分：上下文无关语法 (CFG) 的解析方法

1. 两种基本策略:

○ 自顶向下解析 (Top-down Parsing):

- **思路:** 从起始符号 S 开始, 不断应用产生式规则, 试图生成出与输入句子匹配的终结符序列。
- **特点: 预测驱动 (Prediction-driven).** 先预测结构, 再匹配单词。
- **问题:** 可能会生成大量与输入无关的子树; 左递归规则会导致无限循环。

○ 自底向上解析 (Bottom-up Parsing):

- **思路:** 从输入句子的单词开始, 不断将匹配产生式右侧的序列规约 (reduce) 为左侧的非终结符, 直到最终规约为起始符号 S 。
- **特点: 数据驱动 (Data-driven).** 从单词出发, 构建结构。
- **问题:** 可能会构建很多最终无法连接到 S 的无用成分。

2. 基本解析算法:

○ Shift-Reduce 解析 (移进-规约解析):

- **类型:** 自底向上。
- **数据结构:** 一个栈 (Stack) 和一个输入缓冲区 (Input Buffer)。
- **四种操作:**
 - **移进 (Shift):** 将输入缓冲区的一个词移入栈顶。
 - **规约 (Reduce):** 当栈顶的符号串匹配某个产生式的右侧时, 用该产生式的左侧非终结符替换它们。
 - **接受 (Accept):** 输入缓冲区为空, 且栈中只剩下起始符号 S 。
 - **错误 (Error):** 无法继续进行任何操作。
- **歧义:**
 - **移进-规约冲突 (Shift-Reduce Ambiguity):** 当前状态既可以移进也可以规约。
 - **规约-规约冲突 (Reduce-Reduce Ambiguity):** 栈顶可以被多个不同规则规约。
- **解决:** 通常通过回溯 (Backtracking) 或预设优先级 (如“优先规约”) 来处理。

○ CYK 算法 (Cocke-Kasami-Younger):

- **类型:** 自底向上, 基于动态规划 (Dynamic Programming)。
- **前提:** 语法必须转换为乔姆斯基范式 (Chomsky Normal Form, CNF)。
 - **CNF 规则形式:** $A \rightarrow BC$ 或 $A \rightarrow a$ (右侧要么是两个非终结符, 要么是一个终结符)。
- **过程:** 使用一个 $n \times n$ 的表格 (chart), $Cell[i, j]$ 存储所有可以覆盖从 $word_i$ 到 $word_j$ 的子串的非终结符。自底向上填充表格。
- **复杂度:** $O(n^3)$ 。
- **特点:** 能找出所有可能的分析树, 但本身不具备消歧能力。

第四部分: 统计句法分析 (Statistical Parsing)

1. 动机:

- 传统 CFG 无法解决句法歧义, 一个句子可能有多个合法的分析树。
- 统计模型为每棵分析树分配一个概率, 从而选出最可能的分析树。

2. 概率上下文无关语法 (Probabilistic CFG, PCFG):

- **定义:** 在 CFG 的基础上, 为每条产生式规则 $A \rightarrow \alpha$ 赋予一个概率 $P(A \rightarrow \alpha)$ 。
- **约束:** 对于同一个非终结符 A , 其所有产生式的概率之和为 1 ($\sum_{\alpha} P(A \rightarrow \alpha) = 1$)。
- **分析树概率:** 整棵树的概率是其用到的所有产生式规则概率的乘积。
- **三大假设:**
 1. **位置不变性 (Place Invariance):** 规则概率与在句子中的位置无关。
 2. **上下文无关性 (Context-free):** 子树的生成与其上下文无关。
 3. **祖先无关性 (Ancestor-free):** 子树的生成与祖先节点无关。
- 3. **PCFG 的三个任务 (类比 HMM):**
 - **似然度计算 (Observation Likelihood):** $P(W|G)$ - 计算一个句子 W 由语法 G 生成的概率。
 - **算法: Inside 算法** (对应 HMM 的前向算法)。
 - **最可能推导 (Most Likely Derivation):** $\arg\max_T P(T|W, G)$ - 找出一个句子的最可能分析树。
 - **算法: 概率 CYK 算法 或 PCFG 的 Viterbi 算法** (对应 HMM 的 Viterbi)。
 - **训练 (Training):** $\arg\max_G P(W|G)$ - 从数据中学习 PCFG 的规则概率。
 - **有监督: 从树库 (Treebank) 中直接统计规则频率。** $P(A \rightarrow \beta) = \text{Count}(A \rightarrow \beta) / \text{Count}(A)$ 。
 - **无监督: 从无标注文本中学习。**
 - **算法: Inside-Outside 算法** (对应 HMM 的 Baum-Welch 算法, 是 EM 算法的特例)。
- 4. **PCFG 的局限:**
 - **词汇不敏感:** 规则概率不依赖具体词汇, 导致无法解决需要词汇信息的歧义。例如, 对于 "ate with fork" vs. "ate with meatballs", PCFG 无法区分哪个介词短语挂靠更合理。

第五部分: 词汇化 PCFG (Lexicalized PCFG)

1. **动机:** 解决 PCFG 的词汇不敏感问题。
2. **核心思想:**
 - 为每个短语 (非终结符) 确定一个**核心词 (headword)**。
 - 将核心词信息加入到产生式规则中。
 - 例: $S \rightarrow NP VP$ 变为 $S(\text{ate}) \rightarrow NP(\text{John}) VP(\text{ate})$ 。
3. **挑战:**
 - 规则数量爆炸, 导致**数据稀疏**问题, 难以准确估计概率。
4. **解决方法 (e.g., Collins Parser):**
 - 将产生式分解为**核心词生成**和**左右修饰语生成**的级联过程。
 - 使用平滑技术 (如线性插值) 来估计稀疏规则的概率。

第六部分: 依存句法分析 (Dependency Parsing)

这部分在 Lecture 8 的 PPT 中也大量出现, 与之前总结的知识点一致, 这里再次强调。

1. **核心思想:** 句子结构由词与词之间的**依存关系 (dependencies)** 构成, 每个关系连接一个**核心词 (head)** 和一个**从属词 (dependent)**。
2. **与成分句法的对比:** 依存句法更关注词与词之间的直接关系, 而成分句法关注短语的层级结构。
3. **解析方法:**
 - **基于图的方法 (Graph-based):** 将问题看作是在所有可能的依存弧构成的

图中，寻找一个**最大生成树 (Maximum Spanning Tree)**。全局优化，但速度较慢。

- **基于转移的方法 (Transition-based):** 【重点】模拟一个状态机 (类似 Shift-Reduce)，通过一系列动作 (如 SHIFT, LEFT-ARC, RIGHT-ARC) 来构建依存树。
 - **特点:** 局部决策，贪心搜索，速度快。
 - **Chen & Manning (2014) 的工作:** 首次成功地将**神经网络**引入基于转移的依存分析器，通过学习密集的特征表示，自动组合特征，显著提升了性能和速度，是该领域的里程碑式工作。

第七部分：句法分析评测

1. 成分句法评测 (PARSEVAL):

- **指标:**
 - **精确率 (Precision):** 系统输出的正确成分数 / 系统输出的总成分数。
 - **召回率 (Recall):** 系统输出的正确成分数 / 黄金标准中的总成分数。
 - **F1 值:** 精确率和召回率的调和平均数。
- **带标签 (Labeled) vs. 不带标签 (Unlabeled):** 带标签的评测要求成分的标签 (如 NP, VP) 也必须正确。

2. 依存句法评测:

- **UAS (Unlabeled Attachment Score):** 依存弧的**依附关系** (head-dependent 对) 正确的比例。
- **LAS (Labeled Attachment Score):** **依附关系**和**依存标签** (如 nsubj, dobj) 都正确的比例。**LAS** 是目前最常用、最重要的评测标准。

Lecture 13: Parsing II - 核心知识点总结

第一部分：概率上下文无关语法 (Probabilistic Context-Free Grammar, PCFG)

1. 动机 (Why PCFG?):

- **解决歧义 (Resolving Ambiguity):** 标准的 CFG 可以为一个句子生成多个合法的分析树，但无法判断哪一个更合理。PCFG 通过为每棵树分配一个**概率**，来选择**最可能 (most probable)** 的那棵树。

2. 定义:

- PCFG 是 CFG 的概率化版本，每条产生式规则 $A \rightarrow \alpha$ 都被赋予一个概率 $P(A \rightarrow \alpha)$ 。
- **概率约束:** 对于任何非终结符 A ，其所有产生式规则的概率之和必须为 1。 $\sum_{\alpha} P(A \rightarrow \alpha) = 1$ 。
- **分析树概率:** 一棵分析树的概率是构成该树的所有**产生式规则概率的乘积**。

3. 核心假设 (Assumptions):

- **位置不变性 (Place invariance):** 规则的概率与其在句子中的位置无关。
- **上下文无关性 (Context-free):** 一个子树的生成与其周围的子树无关。
- **祖先无关性 (Ancestor-free):** 一个子树的生成与其祖先节点无关。

4. PCFG 的三大任务 (Analogous to HMM's three problems): 【高频考点】

- **似然度计算 (Observation Likelihood):** $P(W|G)$ - 计算一个句子 W 由语法 G

生成的总概率。

- **算法: Inside 算法** (动态规划, 是 HMM 中前向算法的对应)。
 - **解码/最可能推导 (Most Likely Derivation):** $\text{argmax}_T P(T|W, G)$ - 找到一个句子的最可能分析树。
 - **算法: 概率 CKY 算法** (在 CKY 的表格中存储概率而非仅存储非终结符), 或称为 PCFG 的 Viterbi 算法 (是 HMM 中维特比算法的对应)。
 - **训练/参数估计 (Maximum Likelihood Training):** $\text{argmax}_G P(W|G)$ - 从数据中学习语法规则及其概率。
 - **有监督 (Supervised):** 数据是树库 (Treebank)。直接用最大似然估计 (即统计频率) 计算概率: $P(A \rightarrow \beta) = \text{Count}(A \rightarrow \beta) / \text{Count}(A)$ 。
 - **无监督 (Unsupervised):** 数据是无标注的纯文本。
 - **算法: Inside-Outside 算法** (是 EM 算法的实例, 对应 HMM 中的 Baum-Welch 算法)。
5. PCFG 的优缺点:
- **优点 (Pros):**
 - 能有效解决结构性歧义。
 - 通过剪除低概率路径, 可以加速分析过程。
 - **缺点 (Cons):**
 - **对词汇不敏感 (Not lexicalized):** 规则 $VP \rightarrow V NP$ 的概率不依赖于具体的动词或名词。这导致它无法解决依赖词汇语义的歧义, 如介词短语挂靠问题 ("ate with a fork" vs. "ate with meatballs")。

第二部分: 词汇化 PCFG (Lexicalized PCFG)

1. **动机:** 解决标准 PCFG 对词汇不敏感的问题。
2. **核心思想:**
 - 为每个非终结符 (短语) 关联一个**核心词 (headword)**。
 - 产生式规则中包含核心词信息, 如 $VP(\text{ate}) \rightarrow V(\text{ate}) NP(\text{fish})$ 。
 - **核心词确定规则:**
 - VP 的核心词是主要动词。
 - NP 的核心词是主要名词。
 - PP 的核心词是介词。
3. **挑战与解决方案 (Collins Parser):**
 - **挑战:** 规则词汇化后, 规则数量剧增, 导致数据稀疏问题, 难以准确估计概率。
 - **解决方案:** 将一条复杂的词汇化规则分解为多个生成步骤, 并分别建模其概率。例如, 先生成核心词, 再从核心词向左、向右依次生成修饰语, 直到生成 STOP 符号。这种方法使得参数估计变得可行。

第三部分: 依存句法与解析 (Dependency Syntax and Parsing)

1. **基本概念 (Recap):**
 - 句子结构由词与词之间的**依存关系** (有向弧) 构成。
 - 关系连接**核心词 (head / governor)** 和**从属词 (dependent / subordinate)**。
 - **重要约束: 非交叉约束 (Non-crossing constraint)**, 在很多语言中, 依存弧线不应交叉 (称为投射性 Projectivity)。
2. **解析方法分类:**
 - **基于语法 (Grammar-based):** 如将依存语法转为 CFG 后用 CKY 等算法解

析。

- **数据驱动 (Data-driven):**
 - **基于图 (Graph-based):**
 - **思路:** 将解析看作在所有可能的依存弧构成的图中, 寻找一个**最大生成树 (Maximum Spanning Tree, MST)**。
 - **特点:** 全局优化, 但计算复杂度高, 搜索空间大。
 - **基于转移 (Transition-based): 【现代解析器主流】**
 - **思路:** 定义一个状态机, 通过一系列离散的动作 (transitions) (如 SHIFT, LEFT-ARC, RIGHT-ARC) 来确定性构建依存树。
 - **特点:** 局部决策, 通常是**贪心 (greedy)** 的, 速度快。

第四部分: 神经句法分析 (Neural-based Syntactic Parsing)

1. **核心思想:** 用神经网络自动学习特征, 替代传统方法中复杂的手工特征工程。
2. **成分句法分析 (Constituency Parsing):**
 - **Seq2Tree / Encoder-Decoder:** 使用 Encoder-Decoder 架构, 其中 Decoder 被设计为能生成树状结构。
 - **解析即序列标注:** 将树结构线性化为一个标签序列, 然后用序列标注模型 (如 BiLSTM) 来解决。例如, 标签可以编码 "当前词与下一个词的最低共同祖先节点" 信息。
3. **依存句法分析 (Dependency Parsing) - Chen & Manning (2014) 模型: 【里程碑式工作】**
 - **类型:** 基于转移 (Transition-based) 的神经依存分析器。
 - **革命性贡献:**
 1. **输入:** 不再使用大量离散、稀疏的手工组合特征, 而是使用**低维、密集的词向量、词性向量和标签向量**。
 2. **特征组合:** 通过一个简单的**单隐层神经网络**自动学习特征的有效组合。
 3. **输出:** Softmax 层预测下一个最优的转移动作。
 - **影响:** 证明了简单的神经网络模型配合稠密表示, 可以在**准确率和速度**上双双超越当时复杂的、基于特征的系统。

第五部分: 句法分析评测 (Parsing Evaluation Metrics) - 【必考】

1. **成分句法评测 (PARSEVAL):**
 - **Precision:** $(\# \text{ correct constituents in } P) / (\# \text{ constituents in } P)$
 - **Recall:** $(\# \text{ correct constituents in } P) / (\# \text{ constituents in } T)$
 - **F1-Score:** Precision 和 Recall 的调和平均值。
 - **Labeled vs. Unlabeled:** Labeled 要求成分的跨度 (span) 和标签 (label, 如 NP/VP) 都正确才算正确。
2. **依存句法评测:**
 - **UAS (Unlabeled Attachment Score):** 预测的**核心词 (head)** 正确的词的比例。
 - **LAS (Labeled Attachment Score):** 预测的**核心词 (head)** 和**依存关系标签 (dependency label)** 都正确的词的比例。LAS 是当前依存分析最常用、最重要的标准。

Lecture 10: Text Categorization - 核心知识点总结

第一部分：文本分类基础 (Text Categorization Basics)

1. 定义与目的:
 - 目的: 将自然语言文本 (文档) 自动分配到一个或多个预定义好的类别 (predefined labels) 中。
 - 示例: 垃圾邮件过滤、新闻主题分类 (体育、财经)、情感分析 (正面、负面)、作者识别。
2. 形式化定义:
 - 给定 (Given):
 - 一个实例 (文档) x 。
 - 一个固定的类别集合 $C = \{c_1, c_2, \dots, c_n\}$ 。
 - 确定 (Determine):
 - 实例 x 的类别 $c(x)$, 其中 $c(x)$ 属于 C 。
3. 分类类型 (Classification Types):
 - 文档归属 (Document Membership):
 - 二元分类 (Binary Classification): 只有两个类别 (是/否), 如垃圾邮件检测。
 - 多类分类 (Multi-class Classification): 类别超过两个, 且每个文档只属于一个类别, 如新闻主题分类。
 - 多标签分类 (Multi-labels Classification): 每个文档可以同时属于多个类别。
 - 分类器类型:
 - 硬分类器 (Hard Classifier): 直接给出确定的类别决策。
 - 排序分类器 (Ranking Classifier): 给出文档属于每个类别的概率或得分, 然后进行排序。
4. 学习范式:
 - 监督学习 (Supervised Learning): 主流方法。使用大量已标注的文档进行训练, 学习一个分类器 γ 。
 - 无监督学习 (Unsupervised Learning): 无需标注数据, 系统自动将文档聚类 (cluster)。这通常被称为文本聚类 (Text Clustering)。

第二部分：经典文本分类框架 (Framework of a Classical Text Categorizer)

这是一个非常重要的流程图, 涵盖了从数据到模型评估的全过程。

1. 文档表示 (Document Representation):
 - 核心思想: 将非结构化的文本转换为结构化的向量 (vector)。
 - 词袋模型 (Bag of Words, BoW): 最简单的方法。将文档看作一堆词的集合, 忽略其语法和顺序。
 - 预处理 (Preprocessing):
 - 分词 (Tokenization): 将文本切分成词。
 - 停用词移除 (Stop word removal): 移除 "a", "the", "and" 等功能性词。
 - 词干提取 (Stemming) / 词形还原 (Lemmatization): 将词还原为其基本形式。
2. 特征权重 (Feature Weighting):
 - 核心思想: 为向量中的每个词 (特征) 赋予一个权重, 表示其重要性。

- **TF (Term Frequency, 词频)**: 一个词在文档中出现的次数。词频越高, 越能代表该文档。
 - **IDF (Inverse Document Frequency, 逆文档频率)**: $\log(N / N_k)$, 其中 N 是总文档数, N_k 是包含词 k 的文档数。一个词在越多的文档中出现, 其区分能力越弱, IDF 值越低。
 - **TF-IDF**: $TF * IDF$ 。最经典、最常用的文本特征权重计算方法。它能够有效衡量一个词对于一篇文档的重要程度。
3. **降维 (Dimensionality Reduction)**:
- **动机**: 词汇表通常非常大 (高维), 会导致过拟合、计算效率低等问题。
 - **特征选择 (Feature Selection)**: 从原始特征集中选择一个信息量最大的子集。
 - **方法**: 卡方检验 (Chi-Square), 互信息 (Mutual Information), 信息增益 (Information Gain)。
 - **特征生成/提取 (Feature Generation/Extraction)**: 将原始特征组合或变换为新的、维度更低的特征集。
 - **LSI (Latent Semantic Indexing, 潜在语义索引)**:
 - **思想**: 通过奇异值分解 (SVD) 发现词与文档之间的“潜在语义”关系, 解决同义词 (synonymy) 和多义词 (polysemy) 问题。
 - **方法**: 将词-文档矩阵分解, 并进行低秩近似, 将词和文档映射到一个低维的“潜在语义空间”。
 - **ESA (Explicit Semantic Analysis, 显式语义分析)**:
 - **思想**: 不寻找“潜在”概念, 而是使用一个人类组织的知识库 (如维基百科) 作为“显式”概念集。
 - **方法**: 将文本表示为它与维基百科中每个概念 (文章) 的相关度向量。

第三部分: 文本分类算法 (Text Categorization Algorithms)

1. **Rocchio 算法**:
- **思想**: 基于相关反馈 (Relevance Feedback) 的思想。为每个类别计算一个原型向量 (prototype vector), 即该类别下所有训练文档向量的质心 (centroid)。
 - **分类**: 将新文档分配给与其余弦相似度 (cosine similarity) 最高的那个原型向量所代表的类别。
 - **特点**: 简单、高效, 但只适用于类别能被一个质心很好代表的情况。
2. **朴素贝叶斯 (Naïve Bayes)**:
- **思想**: 基于贝叶斯定理, 计算给定文档 d 的条件下, 属于类别 c 的后验概率 $P(c|d)$, 并选择概率最大的类别。
 - **公式**: $c = \operatorname{argmax}_c P(c|d) = \operatorname{argmax}_c P(d|c)P(c)$
 - **朴素假设 (Naïve Assumption)**: 特征条件独立假设, 即假设文档中的所有词在给定类别 c 的条件下是相互独立的。 $P(d|c) = \prod P(t_i|c)$ 。
 - **问题与解决**:
 - **零概率问题**: 测试集中出现训练集未见过的词, 导致概率为 0。
 - **平滑 (Smoothing)**: 使用拉普拉斯平滑 (Laplace Smoothing) 或加一平滑来解决, 即为所有词的计数加 1。
3. **K-最近邻 (K-Nearest Neighbor, KNN)**:

- **思想: 懒惰学习 (Lazy Learning)**。不建立显式的模型, 训练阶段只是存储所有训练样本。
 - **分类:** 对于一个新文档, 计算它与所有训练文档的相似度, 找到**最相似的 K 个邻居**, 然后通过**多数投票 (majority voting)** 决定其类别。
 - **关键要素:**
 - **相似度度量:** 通常使用余弦相似度。
 - **K 值的选择:** 通过交叉验证等方式凭经验确定。
 - **优缺点:**
 - **优点:** 简单有效, 非线性, 易于更新。
 - **缺点:** 预测慢 (需要计算与所有样本的距离), 对存储要求高。
4. **支持向量机 (SVM):**
- **思想:** 找到一个能将两类数据分开的**最大间隔超平面 (maximum margin hyperplane)**。
 - **特点:** 在高维稀疏的文本数据上表现**非常出色**, 是传统方法中的 SOTA (State-of-the-Art)。
 - **优点:** 理论基础完善, 能有效处理高维数据。
 - **缺点:** 计算复杂度高, 不适合超大规模数据集; 原生只支持二分类。
5. **深度学习方法:**
- **TextCNN:** 利用卷积捕捉局部 n-gram 特征, 通过最大池化聚合信息, 适合分类任务。
 - **RCNN (Recurrent Convolutional Neural Networks):** 结合 BiLSTM 和最大池化, 同时利用上下文信息和关键特征。
 - **预训练模型 (Pretraining Methods):** **【当前 SOTA】**
 - **BERT (及其变体):** 通过**预训练 + 微调 (Pre-training + Fine-tuning)** 的范式解决文本分类。
 - **微调方法:** 通常在 BERT 输出的 [CLS] token 的表示上接一个简单的分类层。
 - **T5:** 将所有 NLP 任务统一为**文本到文本 (Text-to-Text)** 的形式。对于分类, 输入是 "classify: {sentence}", 输出是类别名称的文本。

第四部分: 评测 (Evaluation)

1. **混淆矩阵 (Confusion Matrix):**
 - **TP (True Positive):** 正确预测为正类。
 - **FP (False Positive):** 错误预测为正类。
 - **TN (True Negative):** 正确预测为负类。
 - **FN (False Negative):** 错误预测为负类。
2. **核心评测指标: 【必考】**
 - **准确率 (Accuracy):** $(TP+TN) / (TP+FP+TN+FN)$ 。在类别不平衡时具有误导性。
 - **精确率 (Precision):** $TP / (TP+FP)$ 。在所有被预测为正类的样本中, 真正是正类的比例。
 - **召回率 (Recall):** $TP / (TP+FN)$ 。在所有真正的正类样本中, 被正确预测出来的比例。
 - **F1 值 (F-measure):** $2 * P * R / (P + R)$ 。精确率和召回率的调和平均数, 是最常用的综合评价指标。

3. 多分类评测:
 - 宏平均 (Macro-averaging): 分别计算每个类别的 P/R/F1, 然后求平均。平等对待每个类别。
 - 微平均 (Micro-averaging): 将所有类别的 TP, FP, FN 汇总后, 再计算一个总的 P/R/F1。受样本量大的类别影响更大。
4. 数据集划分:
 - 训练集 (Training set): 用于训练模型。
 - 开发集/验证集 (Development/Validation set): 用于调整超参数。
 - 测试集 (Test set): 用于最终报告模型性能, 在整个过程中只能使用一次。
 - 交叉验证 (Cross-validation): 在数据量较少时, 将训练集划分为 K 份, 轮流使用其中一份作为验证集, 其余作为训练集, 最后将结果平均。

Lecture 11: Text Clustering and Topic Model - 核心知识点总结

第一部分: 文本聚类 (Text Clustering)

1. 定义与目标:
 - 定义: 聚类是一种无监督学习 (unsupervised learning) 方法, 它将未标注的样本集划分为若干个不相交的子集, 称为簇 (clusters)。
 - 目标: 使得簇内相似度高 (high intra-cluster similarity), 簇间相似度低 (low inter-cluster similarity)。
 - 目的: 发现数据中潜在的、自然的分组 (natural groupings), 用于数据组织、知识发现、数据预处理等。
2. 聚类算法两大类型: 【高频对比考点】
 - 层次聚类 (Hierarchical Clustering):
 - 特点: 创建一个嵌套的簇的层次结构, 最终形成一个树状图 (dendrogram)。
 - 凝聚型/自底向上 (Agglomerative / Bottom-up): 最常见。开始时每个样本自成一簇, 然后迭代地合并最相似的两个簇, 直到只剩一个簇。
 - 分裂型/自顶向下 (Divisive / Top-down): 开始时所有样本都在一个簇, 然后迭代地分裂簇, 直到每个样本都自成一簇。
 - 划分式聚类 (Partitional Clustering):
 - 特点: 直接将数据划分为预先指定数量 (k) 的簇。
 - 代表算法: K-Means。

第二部分: 层次凝聚聚类 (Hierarchical Agglomerative Clustering, HAC)

1. 基本算法流程:
 1. 将每个样本初始化为一个单独的簇。
 2. 循环直到只剩一个簇:
 - a. 计算所有簇两两之间的相似度。
 - b. 合并 (merge) 最相似的两个簇 c_i 和 c_j 。
2. 簇相似度计算方法 (Cluster Similarity): 【核心考点】
 - 单链接 (Single Link):
 - 定义: 两个簇的相似度由它们之间最相似的两个样本决

定 $\max(\text{sim}(x,y))$ 。

- **特点:** 容易产生“链式效应 (chaining effect)”，导致细长的、“松散”的簇。

- **全链接 (Complete Link):**

- **定义:** 两个簇的相似度由它们之间 **最不相似** 的两个样本决定 $\min(\text{sim}(x,y))$ 。
- **特点:** 倾向于生成更“紧凑”的、球形的簇，通常效果更好。

- **组平均 (Group Average):**

- **定义:** 计算两个簇中**所有**样本对的平均相似度。
- **特点:** 是单链接和全链接的一种折衷。

3. **计算复杂度:**

- HAC 的基本复杂度是 $O(n^2)$ ，因为需要计算 n 个样本两两之间的相似度。对于大数据集来说效率较低。

第三部分：划分式聚类 - K-均值 (K-Means)

1. **核心思想:**

- 将数据划分为 k 个簇，使得每个样本都属于离它最近的**质心 (centroid)** 所代表的簇。
- **质心:** 一个簇中所有样本向量的**均值**。

2. **K-Means 算法流程:**

1. **初始化:** 随机选择 k 个样本作为初始的簇质心 (种子)。
2. **循环直到收敛:**
 - a. **分配步骤 (Assignment Step):** 对于每个样本，计算它到 k 个质心的距离，并将其分配给最近的那个质心所在的簇。
 - b. **更新步骤 (Update Step):** 重新计算每个簇的质心 (即簇内所有样本的均值)。
 - **收敛条件:** 簇的分配不再改变，或达到最大迭代次数。

3. **关键要素:**

- **目标函数:** 最小化所有样本到其所属簇质心的平方距离之和 (Sum of Squared Errors, SSE)。
- **距离度量 (Distance Metrics):** 欧氏距离 (Euclidean distance, L2-norm)、曼哈顿距离 (L1-norm)、余弦距离 (1 - Cosine Similarity)。
- **NP-hard 问题:** 找到全局最优解是 NP 难的, K-Means 只能保证收敛到**局部**最优解。

4. **K-Means 的弱点及改进:**

- **弱点:**
 - 对**初始种子**的选择非常敏感，可能导致差的聚类结果。
 - 必须**预先指定簇的数量 k** 。
 - 对**异常值 (outliers)** 敏感。
 - 只能发现**球形**的簇。
- **改进 - K-Means++:** 一种更智能的初始化方法，选择的初始种子相互之间尽可能远，能有效改善聚类效果和收敛速度。
- **改进 - Buckshot 算法:** 结合 HAC 和 K-Means。先在一小部分样本上运行 HAC 得到初始种子，再用这些种子去初始化 K-Means，从而避免了随机种子的不稳定性。

第四部分：软聚类与 EM 算法 (Soft Clustering & EM)

1. 硬聚类 vs. 软聚类:
 - 硬聚类 (Hard Clustering): 每个样本被明确地分到一个簇中 (如 K-Means)。
 - 软聚类 (Soft Clustering): 每个样本以一定的概率属于每个簇。
2. 期望最大化算法 (Expectation-Maximization, EM):
 - 用途: 用于含有隐变量 (latent variables) 的概率模型的参数估计, 是软聚类的经典方法。
 - 思想: 迭代地进行两步操作。
 - E 步 (Expectation): 基于当前的模型参数, 计算每个样本属于各个簇的后验概率 (期望)。
 - M 步 (Maximization): 基于 E 步得到的概率, 重新估计模型参数, 以最大化期望似然。
 - Naïve Bayes + EM: 在文本聚类中, 通常假设每个簇是一个朴素贝叶斯模型, 然后用 EM 算法进行无监督训练。
3. 半监督学习 (Semi-Supervised Learning):
 - 思想: 利用少量已标注数据和大量未标注数据一起进行学习。
 - EM 的应用: 在 EM 算法中, 已标注数据的类别是“冻结 (frozen)”的, 只对未标注数据进行概率的重新分配和学习, 从而引导模型学到更好的分类边界。

第五部分: 基于密度的聚类 (Density-based Clustering)

1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise):
 - 思想: 簇是高密度区域, 由低密度区域分隔开。
 - 核心概念:
 - ϵ -邻域: 一个点 p 的 ϵ 距离范围内的所有点。
 - 核心点 (Core point): ϵ -邻域内至少包含 MinPts 个点的点。
 - 边界点 (Border point): 在核心点的 ϵ -邻域内, 但自身不是核心点。
 - 噪声点 (Noise point/Outlier): 既不是核心点也不是边界点。
 - 优点:
 - 能发现任意形状的簇。
 - 能有效处理噪声。
 - 不需要预先指定 k 值。
 - 缺点:
 - 对密度不均的数据集效果不佳。
 - 对参数 ϵ 和 MinPts 的设置敏感。

第六部分: 主题模型 (Topic Model)

1. 定义与目标:
 - 目标: 从大规模文档集合中自动发现隐藏的主题结构 (topical patterns)。是一种无监督的统计方法。
2. LDA (Latent Dirichlet Allocation, 潜在狄利克雷分配): 【主题模型核心】
 - 核心思想 (生成式故事):
 1. 主题是词的 概率分布。 (e.g., “科技”主题下, “computer”, “software” 概率高)
 2. 文档是主题的 概率分布 (混合)。 (e.g., 一篇报道可能 70% 是 “科技”, 30% 是 “财经”)
 3. 文档中的每个词都是先从文档的主题分布中抽取一个主题, 再从该主题的词分布中抽取一个词。

- **隐变量 (Hidden Variables)**: 我们只能观察到文档中的词, 而主题本身、文档的主题分布、词的主题归属都是需要模型**推断 (infer)** 的。
- **推断方法**:
 - 由于后验概率难以直接计算 (intractable), 需要使用**近似推断**。
 - **吉布斯采样 (Gibbs Sampling)**: 一种 **MCMC** 采样方法, 通过构建马尔可夫链来从后验分布中抽样。
 - **变分推断 (Variational Inference)**: 一种**优化方法**, 用一个简单的、可控的分布 q 去**近似**复杂的后验分布 p , 并通过最小化它们之间的 KL 散度来求解。**通常比采样更快**。
- 3. **现代发展**:
 - 将**预训练模型 (如 BERT)** 与主题模型结合, 利用其强大的上下文表示能力来提升主题的一致性 (**coherence**)。
 - 使用**对抗训练 (Adversarial Training)** 等技术来更好地建模分布。

Lecture 12: Information Extraction (IE) - 核心知识点总结

第一部分：信息抽取概览 (Information Extraction Overview)

1. **定义与目标**:
 - **定义**: 从非结构化 (**unstructured**) 或半结构化的文本中, 抽取**结构化 (structured)** 的、预定义类别的信息。
 - **核心任务**: 识别文本中特定类别的**实体 (entities)**、这些实体之间的**关系 (relations)**, 以及它们参与的**事件 (events)**。
 - **目标**: 将文本信息转化为可供计算机直接处理的**知识库 (knowledge base)** 或数据库记录。
2. **IE vs. IR (信息检索)**:
 - **信息检索 (Information Retrieval, IR)**: 返回**相关文档**。用户输入查询, 系统返回一个文档列表。
 - **信息抽取 (Information Extraction, IE)**: 返回**事实和答案**。系统从文档中抽取**结构化的事实片段**。
3. **IE 的挑战**:
 - 文本表达方式多样, 同一个事实可以用不同方式描述。
 - 需要进行复杂的**归一化 (normalization)**, 如时间、金额等。
 - 存在大量的**歧义 (ambiguity)**。

第二部分：命名实体识别 (Named Entity Recognition, NER)

1. **定义**:
 - IE 的一个核心子任务, 目标是**识别 (find)** 并**分类 (classify)** 文本中具有特定意义的实体。
 - **典型类别**:
 - **PER**: Person (人名)
 - **ORG**: Organization (组织机构名)
 - **LOC**: Location (地名)
 - **GPE**: Geo-Political Entity (地理政治实体, 如国家、省份)

- 以及其他如 TIME, MONEY, FACILITY 等。
- 2. **NER 作为序列标注任务:**
 - 方法: 将 NER 任务看作是句子中的每个词 (token) 打上一个标签的**序列标注 (sequence labeling)** 问题。
 - 标注方案 (Encoding Schemes): **【高频考点】**
 - **BIO:** B- (Beginning), I- (Inside), O (Outside)。B- 标记实体的开始, I- 标记实体的内部。**最常用**。
 - **IO:** 只有 I- 和 O。**无法区分两个同类型的实体紧挨着的情况。**
 - **BMES/BIOES:** B- (Beginning), M- (Middle), E- (Ending), S- (Single) 。更复杂的标注, 能更精确地标识实体边界。
- 3. **NER 评测指标 (Metrics):**
 - 基于**精确匹配 (Exact-match)**, 即**实体边界 (boundaries)** 和**类型 (type)** 都必须完全正确。
 - 使用标准的 **Precision, Recall, F1-score** 进行评测。
- 4. **NER 方法:**
 - **基于规则 (Rule-based):**
 - 依赖手工编写的规则和地名/实体词典 (**Gazetteers**)。
 - **优点:** 在特定领域且词典完备时, **精确率高**。
 - **缺点:** 召回率低, 泛化能力差, 构建成本高。
 - **基于特征的机器学习 (Feature-Based):**
 - **经典模型:** MEMM (**Maximum Entropy Markov Model**), CRF (**Conditional Random Field**)。
 - **流程:** 设计大量语言学特征 -> 训练序列分类器。
 - **常用特征:**
 - 词本身 (Current word)、前后文词 (Context words)
 - 词性 (POS tags)、词块 (Chunks)
 - 词形特征 (Short shapes), 如首字母大写、是否包含数字等。
 - 前一个词的标签 (Previous label)。
 - **深度学习 (Deep Learning): 【当前主流】**
 - **核心架构: Bi-LSTM + CRF。**
 - **Bi-LSTM:** 负责从文本中编码上下文信息, 自动学习特征。
 - **CRF:** 负责对标签序列进行建模, 考虑标签之间的依赖关系 (如 I-PER 不能跟在 B-ORG 后面), 输出全局最优的标签序列。
 - **优势:** 端到端学习, 无需手动设计特征, 性能强大。

第三部分: 关系抽取 (Relation Extraction, RE)

1. **定义:**
 - 识别和分类文本中**实体对 (entity pairs)** 之间的**语义关系**。
 - **输入:** 通常是一对实体和它们所在的句子。
 - **输出:** 预定义的关系类型, 如 (Elon Musk, founder, SpaceX)。
2. **关系类型:**
 - **ACE 语料库:** 定义了如 ROLE, PART, AT, SOCIAL 等关系。
 - **Freebase:** 包含数千种细粒度的关系。

3. RE 方法:

- **手写模式 (Hand-built patterns):**
 - 定义词汇-句法模式, 如 X is the founder of Y。
 - 缺点: 覆盖率低, 无法处理多样的语言表达。
- **监督学习 (Supervised Learning):**
 - 思路: 将其视为一个分类问题, 为每个实体对打上关系标签, 然后训练分类器。
 - 缺点: 标注数据昂贵。
- **半监督/自举 (Semi-supervised / Bootstrapping - e.g., Snowball):**
 - 从少量的种子实例 (seed tuples) 开始, 迭代地:
 1. 在文本中找到包含种子实例的句子, 学习抽取模式。
 2. 用学到的模式去发现新的实体对。
 3. 将新的实体对作为新的种子, 重复上述过程。
- **远程监督 (Distant Supervision): 【重要方法】**
 - 思想: 利用一个已有的知识库 (KB) (如 Freebase) 来自动生成训练数据。
 - 核心假设: 如果一个实体对在 KB 中存在某种关系, 那么所有包含这对实体的句子都在表达这种关系。
 - 优点: 能快速获取大量训练数据。
 - 缺点: 强假设会引入大量噪声 (错误标签)。(e.g., "Elon Musk announced the latest addition to the SpaceX arsenal..." 这句话并不表达 founder 关系。)
- **深度学习方法:**
 - **CNN/RNN:** 使用 CNN 或 RNN 对连接两个实体的最短路径 (句法树上) 或整个句子进行编码, 然后进行分类。
 - **联合抽取 (Jointly model):** 将 NER 和 RE 任务放在一个模型中联合学习, 让任务互相促进。
 - **预训练模型 (e.g., R-BERT):** 利用 BERT 等预训练模型强大的表示能力, 显著提升关系抽取性能。

第四部分: 事件抽取 (Event Extraction)

1. 定义:

- 识别文本中发生的事件 (event), 并抽取出其类型、参与者 (participants) 和属性 (attributes)。
- 通常回答 5W1H (Who, When, Where, What, Why, How)。

2. 事件结构 (Event Structure - ACE 定义):

- **事件触发词 (Event Trigger):** 最能表示事件发生的核心词 (通常是动词或名词)。
- **事件论元 (Event Argument):** 事件的参与者 (实体、时间、值等)。
- **论元角色 (Argument Role):** 论元在事件中扮演的角色 (如 Attacker, Target)。

3. 事件抽取类型:

- **封闭域 (Closed-domain):** 事件类型和结构是预定义好的, 任务是填充模板 (template filling)。
- **开放域 (Open-domain):** 不预定义事件类型, 从文本中自动发现事件并进

行聚类（如 TDT 任务）。

4. 事件关系分类 (Event Relation Classification):

- 识别事件之间的关系，如共指 (Coreference)、时序 (Temporal)、因果 (Causality)、子事件 (Subevent)。

第五部分：时间表达式抽取 (Extracting Times)

1. 定义: 作为一个特殊的 NER 任务，专门识别和处理时间信息。

2. 两步法:

1. 时间表达式抽取 (Temporal Expression Extraction):

- 识别文本中的时间短语，如 "yesterday", "April 24, 1916", "two weeks from yesterday"。
- 可使用规则或序列标注方法。

2. 时间表达式归一化 (Temporal Normalization):

- 将抽取出的时间表达式转换为一个标准格式，如 ISO 8601 (YYYY-MM-DDTHH:MM:SS)。
- 需要处理绝对时间、相对时间和持续时间，并依赖于参考时间 (如文章发布时间 DCT)。

Lecture 13: Natural Language Generation (NLG) - 核心知识点总结

第一部分：NLG 基础与解码算法

1. NLG 定义:

- 广义上指任何生成新文本的任务。
- 是许多下游任务的核心组成部分，如机器翻译、摘要、对话、故事生成等。

2. 语言模型 (Language Model, LM):

- 任务: 预测给定上文的条件下，下一个词是什么。即计算概率分布 $P(y_t | y_1, \dots, y_{t-1})$ 。
- 条件语言模型 (Conditional LM): 在预测下一个词时，不仅考虑上文，还考虑一个额外的输入 x 。即计算 $P(y_t | y_1, \dots, y_{t-1}, x)$ 。这是大多数 NLG 任务的基础（如翻译、摘要）。

3. 解码算法 (Decoding Algorithms): 【NLG 核心考点】

- 问题: 训练好的语言模型如何用来实际生成文本？
- 定义: 解码算法是从语言模型（一个概率分布）中生成文本序列的策略。

4. 常见解码算法:

○ 贪心解码 (Greedy Decoding):

- 策略: 在每一步，都选择当前概率最高 (argmax) 的那个词作为输出。
- 缺点: 简单但效果差。容易陷入局部最优，生成重复、不连贯的文本。

○ 集束搜索 (Beam Search Decoding):

- 核心思想: 在每一步，不再只保留一个最优选择，而是保留 k 个概率最高的部分序列 (hypotheses)。 k 称为 beam size。
- 流程: 从这 k 个候选项出发，扩展下一步，再从所有可能的扩展中选出新的 k 个最优候选项，如此往复。
- 优点: 比贪心效果好得多，能找到概率更高的序列。

- 缺点/权衡:
 - 小 k: 效果接近贪心。
 - 大 k: 计算成本高; 对于某些任务 (如翻译), k 过大会导致生成的句子过短、过于常见 (**generic**), 反而降低评测指标 (如 BLEU)。
- 采样解码 (Sampling-based Decoding):
 - 动机: 增加生成文本的多样性 (**diversity**) 和随机性 (**randomness**)。
 - 纯采样 (Pure Sampling): 在每一步, 直接从模型的整个概率分布中随机抽样一个词。
 - Top-n (Top-k) 采样:
 - 策略: 将概率分布截断 (**truncate**), 只在概率最高的 n 个词中进行采样。
 - 效果: n=1 等价于贪心搜索; n 越大, 越接近纯采样, 生成文本越多样/冒险; n 越小, 生成文本越保守/安全。
- 温度采样 (Softmax Temperature):
 - 注意: 它不是一个解码算法, 而是一种调节概率分布的技术, 需要配合采样或 Beam Search 使用。
 - 公式: 在 Softmax 函数中引入温度参数 τ : $\exp(s_w / \tau)$ 。
 - 效果:
 - $\tau > 1$ (提高温度): 概率分布变得更平坦 (**uniform**), 增加了低概率词被选中的机会, 生成结果更多样。
 - $\tau < 1$ (降低温度): 概率分布变得更尖锐 (**spiky**), 模型更倾向于选择高概率词, 生成结果更保守。

第二部分: NLG 核心任务与方法

1. 任务定义:
 - 单文档摘要 (Single-document) / 多文档摘要 (Multi-document)。
2. 两大方法: 【高频对比考点】
 - 抽取式摘要 (Extractive Summarization):
 - 思想: 从原文中选择关键的句子或短语, 直接组合成摘要。
 - 优点: 语法正确, 事实准确, 实现相对简单。
 - 缺点: 不够流畅, 可能存在冗余, 灵活性差 (**no paraphrasing**)。
 - 生成式摘要 (Abstractive Summarization):
 - 思想: 理解原文内容, 然后用模型自己的话生成新的文本作为摘要。
 - 优点: 更像人类, 流畅、简洁, 灵活性高。
 - 缺点: 技术上更困难, 可能产生事实错误 (幻觉 Hallucination)、重复等问题。
3. 摘要方法演进:
 - 前神经网络时代: 主要是抽取式。
 - 方法: 基于词频、TF-IDF、文本结构 (标题、首句)、图方法 (如 LexRank, 受 PageRank 启发, 认为重要的句子是与其它重要句子连接紧密的句子)。
 - 神经网络时代 (Seq2Seq):
 - 基本模型: 将摘要视为翻译任务, 用 Seq2Seq + Attention 模型。

- **问题与改进:**
 - **问题 1: 事实错误/复制细节差:** Seq2Seq 模型倾向于生成流畅但可能不准确的文本，特别是对于稀有词（如人名）。
 - **解决方案: 拷贝机制 (Copy Mechanisms),** 如 Pointer-Generator Networks。允许模型在生成时直接从原文中**复制**词语，实现了抽取和生成的混合。
 - **问题 2: 拷贝过多:** 拷贝机制可能导致模型退化为纯抽取式。
 - **解决方案: 自底向上摘要 (Bottom-up Summarization)。** 分两步走: 先用一个模型**选出**重要内容（内容选择），再让生成模型在这些选定内容上进行生成（文本实现）。
 - **问题 3: ROUGE 指标与人类感知不符:** 直接用最大似然训练不等于优化 ROUGE。
 - **解决方案: 强化学习 (Reinforcement Learning)。** 将 ROUGE 分数作为奖励信号，直接优化生成模型。**发现:** RL 能提高 ROUGE，但可能牺牲可读性；结合 ML 和 RL 的混合方法效果最好。
 - 1. **分类:**
 - **任务导向型 (Task-oriented):** 帮助用户完成特定任务，如订票、客服。
 - **社交型 (Social dialogue / Chit-chat):** 开放式闲聊。
 - 2. **Seq2Seq 在对话中的问题:**
 - **通用/无聊回复 (Generic/boring responses):** "I don't know", "Me too"。
 - **不相关回复 (Irrelevant responses):** 回答与上下文无关。
 - **重复 (Repetition):** 生成重复的词或短语。
 - **缺乏上下文记忆 (Lack of context)。**
 - **缺乏一致人设 (Lack of consistent persona)。**
 - 3. **解决方案:**
 - **MMI (Maximum Mutual Information):** 在优化目标中加入惩罚项，惩罚那些与输入无关的、过于通用的回复。
 - **检索-精炼 (Retrieve-and-refine):** 先从语料库中检索一个相似的回复，再用模型进行修改和精炼。
 - **人设建模 (Persona Modeling):** 将用户/机器人的“人设”信息作为额外输入编码到模型中。
 - **强化学习:** 在特定任务（如谈判对话）中，将任务成功与否（如谈判得分）作为奖励信号来优化策略。
 - **挑战:** 需要长期的**连贯性 (coherence)**、**规划性 (planning)** 和**结构性 (structure)**，这些是标准语言模型所缺乏的。
 - **方法:**
 - **分步生成:** 先生成故事大纲或骨架，再填充细节。
 - **约束解码:** 在生成时加入外部约束，如诗歌的**格律 (meter)** 和**韵律 (rhyme)**，可以通过有限状态机 (FSA) 来实现。
 - **多任务学习:** 同时学习语言模型、格律模型、韵律模型。
-

第三部分：NLG 评测 (NLG Evaluation) - 一个棘手的难题

1. **自动评测指标 (Automatic Metrics):**

- **基于词重叠 (Word Overlap):**
 - **BLEU:** 基于**精确率 (precision)**, 常用于机器翻译。
 - **ROUGE:** 基于**召回率 (recall)**, 常用于摘要。主要有 ROUGE-N (n-gram) 和 ROUGE-L (最长公共子序列)。
 - **问题:** 这些指标在 NLG 任务中, 尤其是开放式任务 (如对话、故事) 中, 与**人类判断的相关性很差**。一个好的回复可以有多种表达方式。
 - **其他指标:** 困惑度 (Perplexity), 基于词向量的相似度等, 但同样存在问题。
 - 2. **人工评测 (Human Evaluation):**
 - **黄金标准 (Gold Standard)**, 但存在诸多问题:
 - **昂贵且耗时 (Expensive and slow)**。
 - **主观性强 (Subjective)**, 人类评价者之间**不一致 (inconsistent)**。
 - 评价者容易疲劳、误解任务。
 - 3. **未来方向:**
 - **语料库级指标:** 评价整个测试集的生成质量, 而非单个样本。
 - **对抗性评测:** 训练一个判别器来区分机器生成和人类生成的文本。
 - **众包/游戏化:** 设计有趣的任务让用户在不知不觉中完成评测。
- 核心结论:** 评测是 NLG 领域目前最大的瓶颈和挑战。

Lecture 14: Question Answering (QA) - 核心知识点总结

第一部分: 问答系统概览 (QA Overview)

1. **定义与目标:**
 - **目标:** 自动回答用户用**自然语言**提出的问题。
 - 与搜索引擎返回**文档列表**不同, QA 系统旨在提供**直接、精确的答案**。
2. **问题类型:**
 - **事实型问题 (Factoid Questions):** 寻求一个简单事实的答案。答案通常是**命名实体** (人名、地名、时间、数字等)。
 - 例: "中国的首都是哪个城市?"
 - **复杂/叙述性问题 (Complex/Narrative Questions):** 需要综合信息、解释或给出观点。
 - 例: "基于 mRNA 研发的疫苗效果怎么样?"
3. **QA 系统范式 (Paradigms for QA):**
 - **基于信息检索 (IR-based):**
 - **核心思想:** 将问题作为查询, 从大规模文本语料库 (如 Web) 中检索相关文档/段落, 然后从中抽取出答案。
 - **代表:** 早期 Google/Baidu 搜索、IBM Watson 的一部分。
 - **基于知识库 (Knowledge-based):**
 - **核心思想:** 将自然语言问题转换为对**结构化知识库 (Knowledge Base, KB)** (如 DBpedia, Freebase, Wikidata) 的**形式化查询** (如 SQL, SPARQL)。
 - **代表:** Apple Siri, Wolfram Alpha。
 - **混合方法 (Hybrid Approaches):**
 - **核心思想:** 结合 IR 和 KB 两种方法的优势。

- 代表: IBM Watson, 既从文本中检索候选答案, 也利用知识库进行推理和打分。

第二部分: 基于 IR 的事实型问答 (IR-based Factoid QA)

这是一个经典的三阶段流水线模型 (pipeline):

1. 问题处理 (Question Processing):

- 目标: 分析问题, 理解用户的意图。
- 子任务:
 - **答案类型检测 (Answer Type Detection):** 最关键的一步。判断答案期望的**实体类型** (如 PERSON, LOCATION, DATE)。这可以极大地缩小答案的搜索范围。
 - 方法: 可以用手写规则 (如 Who 开头的问题对应 PERSON), 或训练一个分类器。
 - **查询构建 (Query Formulation):** 从问题中提取**关键词**, 构建能够有效检索相关文档的查询。
 - **焦点词检测 (Focus Detection):** 找到问题中可能被答案替换的部分 (如 "the two states")。
 - **关系抽取 (Relation Extraction):** 识别问题中实体间的关系 (如 borders(Florida, ?x, north))。

2. 段落检索 (Passage Retrieval):

- 目标: 找到最有可能包含答案的文本片段。
- 流程:
 1. **文档检索:** 使用问题关键词通过标准 IR 引擎 (如 TF-IDF, BM25) 召回相关文档。
 2. **段落切分:** 将长文档切分为更小的单元 (如段落或句子)。
 3. **段落排序:** 对段落进行重排序, 选出与问题最相关的段落。
 - **排序特征:** 包含的**答案类型实体**数量、**查询词**数量、**查询词邻近度**等。

3. 答案处理/抽取 (Answer Processing/Extraction):

- 目标: 从排序后的段落中**精确定位**并**抽取**出**答案字符串**。
- 流程:
 1. **候选答案抽取:** 在段落中运行一个专门的**命名实体识别器 (NER)**, 其类别与问题处理阶段确定的答案类型相匹配。
 2. **候选答案排序:** 如果有多个候选答案, 使用一个**排序模型**对它们打分。
 - **排序特征:**
 - **答案类型匹配度:** 候选答案与预期类型的匹配程度。
 - **模式匹配:** 是否匹配某些常见的回答模式 (如 X, founder of Y)。
 - **新颖度 (Novelty):** 候选答案中是否包含问题中没有的词。
 - **与问题词的距离:** 候选答案与查询关键词的距离。

第三部分: 基于知识的问答 (Knowledge-based QA)

1. 核心流程:

- **语义解析 (Semantic Parsing):** 将自然语言问题映射为知识库可以理解的**逻辑形式**或**查询语句**。

- **知识库查询**: 在 KB (如 DBpedia, Freebase, Yago) 中执行查询, 获取答案。
- 2. **利用知识库进行推理**:
 - **时序推理 (Temporal Reasoning)**: 利用实体的出生/死亡日期等信息过滤不合理的候选答案。
 - 例: 问 1594 年的事, 出生于 1817 年的梭罗不可能是答案。
 - **地理空间推理 (Geospatial Reasoning)**: 利用地理位置的包含、方向、边界等关系。
 - 例: “亚洲城市”的答案不可能是“巴黎”。

第四部分: 机器阅读理解 (Machine Reading Comprehension, MRC)

1. **定义**:
 - QA 的一个特定形式, 系统被要求在给定的一个或几个段落 (context/passage) 中回答问题。
 - **输入**: (Context, Question)
 - **输出**: Answer
 2. **SQuAD 数据集 (Stanford Question Answering Dataset): 【MRC 里程碑】**
 - **SQuAD 1.1**:
 - **特点**: 答案保证是原文中的一个连续文本片段 (span)。这极大地简化了问题, 使其可以被建模为预测答案的起始位置和结束位置。
 - **评测指标**:
 - **精确匹配 (Exact Match, EM)**: 预测的答案与标准答案完全一致。
 - **F1 分数**: 将预测答案和标准答案看作词袋, 计算词级别的 F1 分数。更常用、更鲁棒。
 - **SQuAD 2.0**:
 - **改进**: 引入了无法回答的问题 (unanswerable questions), 系统不仅要找到答案, 还要判断问题是否能在给定的上下文中被回答。
 - **挑战**: 要求模型具备“知之为知之, 不知为不知”的能力。
 3. **MRC 神经网络模型架构: 【重点】**
 - **基本范式**: 使用两个编码器分别对问题(Question)和上下文(Context)进行编码, 然后通过一个注意力层进行交互, 最后由一个输出层预测答案。
 - **Attentive Reader (Hermann et al., 2015)**: 早期的经典模型, 使用双向 LSTM 和注意力机制。
 - **BiDAF (Bi-Directional Attention Flow, Seo et al., 2017)**:
 - **核心思想**: 注意力应该是双向流动的。
 - **C2Q (Context-to-Question) Attention**: 对于每个上下文词, 计算它与问题中哪些词最相关。
 - **Q2C (Question-to-Context) Attention**: 找出上下文中哪些词对于整个问题最重要。
 - **影响**: 成为后续许多 MRC 模型的标准组件。
 - **现代模型**:
 - BERT 及其变种已完全主导了 MRC 领域。通过将问题和上下文拼接成一个序列输入, 利用 Transformer 强大的上下文编码和交互能力, 直接预测答案的起始和结束位置。
-

第五部分：IBM Watson (混合方法的典范)

- Watson 是一个复杂的**混合系统**，其架构展示了如何整合多种技术来解决困难的 QA 任务（如 Jeopardy!）。
- **高层架构**:
 1. **问题处理**: 进行深度句法和语义分析。
 2. **候选答案生成**:
 - 从**文本资源**中通过 IR 和 IE 技术生成候选。
 - 从**结构化数据**（知识库）中生成候选。
 3. **候选答案打分: 核心步骤**。使用**超过 50 个**不同的打分组件（Scorers）从不同维度为每个候选答案打分，这些维度包括：
 - 与问题句法/逻辑形式的匹配度
 - 来源可靠性
 - 时序/地理空间一致性
 - 分类学一致性
 4. **合并与排序**:
 - **合并等价答案**（如 J.F.K. 和 John F. Kennedy）。
 - 训练一个**排序分类器**（逻辑回归），输入所有打分组件得到的分数向量，输出最终的置信度，并对答案进行排序。

Lecture 14: Large Language Model I - 核心知识点总结

第一部分：预训练模型概览 (Pre-trained Model Overview)

1. **预训练范式 (Pre-training + Fine-tuning Paradigm)**:
 - **核心思想**: 这是由 BERT 和 GPT 等模型开创的现代 NLP 新范式。
 1. **预训练 (Pre-training)**: 在**海量无标注文本**上训练一个通用的**基础模型 (Foundation Model)**，让其学习普适的语言知识。
 2. **微调 (Fine-tuning)**: 将预训练好的模型在**特定下游任务**的有标注数据上进行少量训练，使模型适应具体任务。
 - **三要素**: **大数据 (Big Data)**, **大模型 (Big Models)**, **大算力 (Big Compute Power)**。
2. **预训练模型两大类**:
 - **NLU (Natural Language Understanding) 模型**:
 - **代表**: BERT, RoBERTa, ALBERT, ELMo。
 - **架构**: 通常基于**双向 (Bidirectional)** 的 Encoder 结构（如 Transformer Encoder）。
 - **任务**: 学习深度的上下文表示，适用于**分类、抽取**等理解型任务。
 - **预训练目标**: **MLM (Masked Language Model)**, **NSP (Next Sentence Prediction)** 等。
 - **NLG (Natural Language Generation) 模型**:
 - **代表**: GPT 系列, T5, BART。
 - **架构**:
 - **Decoder-only (自回归)**: GPT 系列。只能看到左侧上下文，适合文本生成。

- **Encoder-Decoder:** T5, BART。既有双向编码器理解输入，又有自回归解码器生成输出。
 - **任务:** 适用于生成式任务，如对话、摘要、翻译。
 - **特点:** 输出灵活性高，但评测困难。
-

第二部分：大模型的“大”与“涌现能力”

1. **大模型趋势 (Moore's Law for Large Models):**
 - 模型参数量和算力需求大约**每年增长 10 倍**，发展速度极快。
 - GPT-2 (2019) -> GPT-3 (2020) -> PaLM (2022)，参数量从亿级增长到数千亿级。
 2. **规模效应与涌现能力 (Scale Effect & Emergent Capability):** **【LLM 核心概念】**
 - **核心观点:** 量变引起质变。当模型规模（参数量、训练数据量）跨越某个临界点后，其能力会**突然涌现 (erupt)**，表现出之前小模型不具备的新能力（如上下文学习、思维链推理）。
 - **能力增长曲线:**
 - **对数增长 (Logarithmic):** 传统任务，性能随规模增长而平滑提升。
 - **突变式增长 (Mutational):** 复杂、挑战性任务，在规模达到某个阈值时性能突然爆发。
 - **反常增长 (Anomalous / Inverse Scaling):** 少数误导性任务，模型变大反而性能下降（但后续通过指令微调等方式可以纠正）。
-

第三部分：预训练数据 (Pre-trained Data)

1. **数据的重要性:**
 - 数据是模型知识的**唯一来源**。数据质量和多样性直接决定了模型的能力上限。
 - **“Garbage in, garbage out”:** 低质量的训练数据会导致模型性能下降。
2. **数据来源与构成:**
 - **来源:** CommonCrawl (海量网页数据)、书籍、维基百科、代码 (GitHub)、社交媒体等。
 - **构成:** 高质量的文本和代码是目前主流 LLM 训练数据的核心。
3. **数据处理 (Data Processing):** **【关键工程实践】**
 - **过滤 (Filtering):**
 - **基于规则/启发式:** 过滤低质量网页、去广告。
 - **基于分类器:** 训练一个分类器来区分高质量和低质量文本。
 - **基于度量:** 使用**困惑度 (Perplexity, PPL)** 过滤不连贯的文本。
 - **去重 (Deduplication):**
 - **目的:** 移除重复数据, 防止模型在重复样本上过拟合, 提高训练效率。
 - **影响:** 去重至关重要。实验表明, 数据重复会严重损害模型的有效参数量, 使**缩放定律 (Scaling Law)** 失效。
 - **隐私信息删除:** 使用规则或模型移除个人隐私数据。
4. **缩放定律 (Scaling Law):**
 - **核心内容:** 模型的性能（通常用 Test Loss 表示）与**模型大小 (N)**、**计算量 (C)** 和**数据量 (D)** 之间存在可预测的**幂律关系 (power-law)**。
 - **意义:** 使得我们可以在小规模实验的基础上，**预测**更大模型在更多数据和算力下的性能，从而指导大模型的训练。

- **数据质量的影响**: 缩放定律成立的前提是高质量、经过精心处理的数据。
-

第四部分：LLM 训练流程 (LLM Training Process)

这是一个标准的三阶段流程，以 InstructGPT / ChatGPT 为代表：

1. **Step 1: 监督微调 (Supervised Fine-Tuning, SFT)**
 - **目标**: 教模型学会遵循指令和进行多轮对话的基础格式。
 - **数据**: 高质量的 {prompt, response} 对。
 - **功能**:
 - **格式学习 (Format Learning)**: 学会对话格式。
 - **能力增强 (Capability Reinforcement)**: 在特定能力（如代码）上进行强化。
 - **自我认知 (Self-awareness)**: 学会说 "I don't know"。
2. **Step 2: 奖励建模 (Reward Modeling, RM)**
 - **目标**: 训练一个奖励模型，使其能够评估模型生成回复的质量，即学会人类的偏好。
 - **数据**: 对同一个 prompt，由 SFT 模型生成多个不同的回复，然后由人工标注员对这些回复进行排序。输入是 {prompt, response_win, response_lose}。
 - **训练**: 奖励模型被训练成一个能给“更好”的回复打更高分的分类器或回归器。
3. **Step 3: 对齐 (Alignment) - 基于人类反馈的强化学习 (RLHF)**
 - **目标**: 使用强化学习来优化 SFT 模型，使其生成的回复能从奖励模型中获得更高的分数，从而使其行为与人类偏好对齐 (align)。
 - **核心算法**: PPO (Proximal Policy Optimization)。
 - **流程**:
 1. 用 SFT 模型（作为策略 Policy）对一个 prompt 生成一个回复。
 2. 用训练好的 RM 给这个回复打分（作为奖励 Reward）。
 3. 用这个奖励信号通过 PPO 算法更新 SFT 模型的参数。
 - **关键约束**: 在 PPO 的目标函数中加入一项 **KL 散度 (KL divergence)** 约束，防止优化后的模型与原始 SFT 模型偏离太远，保持语言能力和多样性。
 - **过优化问题 (Overoptimization)**: 过度优化 RM 可能会导致模型“钻空子 (hacking)”来获得高分，但实际质量下降。
 - **解决方案**: 使用更大的 RM、集成多个 RM、更细粒度的偏好标注（如过程监督 Process-supervised RM）。

第五部分：通用语言模型架构与挑战 (GLM Architecture & Challenges)

- 这部分回顾了 Transformer 架构的各种变体和优化，旨在解决大模型训练中的效率和性能问题。
- **不可能三角 ("Impossible" Triangle)**:
 - **训练效率 (Training efficiency)**: 可并行。
 - **推理效率 (Reasoning efficiency)**: 推理成本不随长度显著增加。
 - **建模能力 (Modeling ability)**: 能处理长距离依赖。
 - **困境**: Transformer 在训练效率和建模能力上强，但推理效率差；RNN 推理效率高，但训练慢、长距离依赖差；线性 Transformer 训练和推理都快，但建模能力弱。
- **架构演进**:

- **Self-attention 优化**: Multi-Query Attention, FlashAttention 等, 旨在降低计算和内存复杂度。
- **Normalization Layer**: 从 Post-LN 到 Pre-LN, 再到 RMSNorm, 以提高训练稳定性。
- **Positional Embedding**: 从绝对位置编码到相对位置编码 (RoPE), 再到处理超长文本的插值方法。
- **新架构探索**: 状态空间模型 (SSM) 如 H3, Mamba, 试图打破不可能三角, 兼顾效率和性能。

第一部分：大语言模型 (LLM) 的演进

这部分详细追溯了从闭源的 GPT 系列到繁荣的开源社区, 再到针对特定语言 (中文) 的适应性开发。

1. GPT 系列演进路线 (闭源模型的标杆)

GPT 系列展示了模型能力随规模和技术迭代而增强的清晰路径。

- **GPT-1 (2018)**: 确立了“**预训练 + 微调**” (Pre-training + Fine-tuning) 的范式, 成为后续研究的基础。
- **GPT-2 (2019)**: 引入 **零样本 (Zero-shot)** 学习能力, 证明了模型在没有特定任务微调的情况下也能完成下游任务。模型规模和数据量显著增加。
- **GPT-3 (2020)**: 参数量达到惊人的 1750 亿, 开启了超大规模模型时代。核心能力是 **少样本 (Few-shot)** 学习, 通过在提示中给出少量示例, 即可完成任务。
- **InstructGPT (2022)**: 关键转折点。引入了 **基于人类反馈的强化学习 (RLHF)**, 使模型输出与人类意图和偏好对齐 (Helpful, Honest, Harmless), 极大提升了实用性。
- **ChatGPT (GPT-3.5) (2022)**: 基于 InstructGPT, 并结合了对话数据进行优化, 引爆了全球对大模型的关注。展示了强大的对话、代码理解和多轮交互能力。
- **GPT-4 (2023)**: 成为新的性能标杆。核心提升包括:
 - **多模态能力**: 支持图像输入。
 - **更强推理能力**: 在各类考试和复杂任务上表现优异。
 - **更长上下文**: 支持高达 32k token 的上下文。
 - **扩展能力**: 通过 **网页浏览 (Web Browsing)** 和 **代码解释器 (Code Interpreter)** 等插件, 与外部工具交互, 弥补了知识时效性和计算能力的不足。

2. 开源大模型的兴起 (社区的追赶与创新)

针对商业模型“黑箱”的问题, 开源社区迅速发展, LLaMA 是其中的里程碑。

- **LLaMA (Meta)**: 虽然参数量 (最大 65B) 远小于 GPT-3, 但通过使用海量的训练数据, 其性能在许多任务上能与 GPT-3 媲美甚至超越。这证明了**数据质量和数量的重要性**。
 - **核心技术**: 采用了 RMSNorm、SwiGLU 激活函数、RoPE 位置编码等高效技术。
- **Alpaca (Stanford)**: 证明了“**小成本也能出奇迹**”。它使用 LLaMA 作为基础模型, 通过调用 GPT-3.5 API 生成约 5 万条指令数据进行微调, 成本不到 600 美元, 就获得了接近 GPT-3.5 的效果。这种“**蒸馏**”或“**数据合成**”的思想影响深远。
- **Llama-2 (Meta)**: LLaMA 的正式开源迭代版本, 可商用。训练数据更多, 上下文窗口

更长，并引入了 **分组查询注意力 (GQA)** 来提升推理速度。

3. 中文大模型的适配

主要以 **Chinese-LLaMA-Alpaca** 项目为例，解决了开源模型在中文处理上的两大痛点。

- **问题 1: 中文词表缺失**
 - **原因:** LLaMA 原生词表几乎不含中文, 导致一个汉字被切分成多个 Byte-level token, 编码效率极低。
 - **解决方案:** 扩充中文词表, 将原生词表与训练好的中文词表合并, 使编码效率翻倍。
- **问题 2: 训练成本高昂**
 - **解决方案:** 采用 **LoRA (Low-Rank Adaptation)** 进行参数高效微调。冻结大部分模型参数, 只训练新增的、小规模的低秩矩阵, 极大降低了微调的硬件门槛和时间成本。

第二部分: LLM 的基准测试与评估

这部分探讨了如何衡量和比较不同 LLM 的能力。

- **评估维度:** 分为 **基础能力** (语言生成、知识利用、复杂推理等) 和 **高级能力** (人类对齐、与环境交互、工具使用等)。
- **评估方法:**
 - **基准测试 (Benchmark):** 在标准化的公开数据集上进行测试, 如 MMLU、HellaSwag 等。
 - **人类评估:** 由人类对模型生成的内容进行主观打分, 如 Chatbot Arena。
- **核心发现:**
 - **闭源模型仍领先:** ChatGPT、Claude 2 等在复杂推理和工具使用方面优势明显, 这得益于其专门的优化和插件生态。
 - **指令微调有效:** 经过指令微调的开源模型 (如 Alpaca) 普遍优于其基座模型 (如 LLaMA)。
 - **开源模型有短板:** 在数学推理、工具使用等复杂任务上, 现有开源模型与顶级闭源模型差距较大。
 - **规模效应显著:** 模型规模越大, 性能通常越好, 尤其是在复杂任务上。

第三部分: LLM 面临的问题与挑战

这部分探讨了当前 LLM 技术的核心难题和未来方向。

1. 幻觉 (Hallucination)

- **定义:** 模型生成与事实不符、无中生有或无法验证的内容。
- **原因:**
 - **数据层面:** 训练数据本身存在噪声或不一致。
 - **模型层面:** 知识存储在参数中导致偏差、解码策略鼓励多样性等。
- **解决方案:**
 - **检索增强生成 (RAG):** 在生成答案前, 先从外部知识库 (如网页、数据库) 检索相关信息, 并以此为依据进行回答, 可以有效提升事实准确性。这正是 GPT-4 网页浏览插件的原理。

2. 伸缩法则 (Scaling Law)

- **定义:** 模型的性能 (Loss) 与 **计算量(C)**、**模型参数量(N)**、**数据量(D)** 这三个因素呈可预测的幂律关系。

- **重要性:** 它使得我们可以在小规模模型上进行实验, 从而预测超大模型的性能, 极大地指导了模型的训练和资源分配。
- **LLaMA 的“反伸缩法则”:** LLaMA 的研究表明, 为了获得 **最佳的推理性能 (即运行成本低)**, 应该用比传统伸缩法则建议的更多的数据去训练一个相对较小的模型。

3. 可扩展对齐 (Scalable Alignment)

- **核心问题:** 基于人类反馈的 RLHF 成本高、效率低、主观性强, 难以扩展。
- **深层挑战:** 当 AI 的能力超越人类时, 我们 (作为较简单的系统) 如何监督和对齐一个更复杂的系统? 这是一个开放性的前沿研究问题。
- **探索方向: RLAIIF (Learning from AI Feedback),** 即用一个更强大的 AI 模型来提供反馈信号, 以训练和对齐另一个 AI 模型。